discopt: A Differentiable Mixed-Integer Nonlinear Programming Solver with a Memory-Safe Interior Point Method

John R. Kitchin

2026-07-18

1 Abstract

We present discopt, a hybrid Rust/JAX/Python solver for mixed-integer nonlinear programming (MINLP) problems, which integrates POUNCE, a separate companion project providing a faithful Rust translation of Ipopt's interior point algorithm, as one of its NLP backends. The system makes six contributions: (1) differentiable MINLP solving via implicit differentiation of the KKT system, enabling decision-focused learning through nonconvex optimization layers; (2) batched node evaluation using `jax.vmap` for data-parallel branch-and-bound; (3) a pure-JAX interior point method with Mehrotra predictor-corrector steps and Ipopt-style filter line search that is JIT-compilable and vmappable; (4) integration of POUNCE, a memory-safe Rust IPM with zero C/Fortran dependencies that implements Ipopt's numerical hardening through a source-level translation; (5) a compositional relaxation compiler supporting McCormick, α BB, piecewise McCormick, and ICNN-learned relaxations; and (6) a neural network embedding module that formulates trained feedforward networks as algebraic MINLP constraints via full-space, big-M, and reduced-space strategies, enabling optimization over ML surrogates. The linear and mixed-integer subproblems throughout — every spatial-B&B node relaxation and the AMP MILP master — are solved by a self-hosted, hardened pure-Rust revised simplex that retires an external-solver fallback chain in favour of in-engine numerical robustness (iterative refinement, condition monitoring, and EXPAND anti-degeneracy). The global solver couples spatial branch and bound with a cutting-plane framework (reformulation-linearization, second-order-cone, and moment/PSD separators under a structure-gated `cuts`'auto'= policy), complementarity/MPEC reformulations, automatic geometric-program detection with log-space convex routing, and irreducible-infeasible-subsystem diagnosis; differentiability extends to mixed-integer programs via a fix-and-differentiate restriction. All three NLP backends (pure-JAX IPM, POUNCE, Ipopt) produce identical optimal solutions on a suite of 9 NLP and 24 MINLP benchmark problems with zero incorrect results. Because a global solver's product is its *certificate*, we report a systematic soundness audit — roughly three dozen tracked correctness fixes plus a twenty-module code review — that hardened the certificate across the relaxation, presolve, branch-and-bound, cut, parser, and dual-bound layers while maintaining a zero-incorrect-result gate. In a head-to-head against full-license BARON[1] and SCIP[2]

over roughly two hundred MINLPLib instances at a matched 60~s budget, discopt records **zero** correctness violations against either solver — no run ever certified a global optimum at the wrong value or returned an incumbent better than the proven optimum; on the orthogonal *coverage* axis (the fraction of instances certified globally optimal within the budget) the two mature solvers lead, each certifying on the order of 180 instances to discopt's $\approx$ 120, a per-node wall-time gap on a serial engine rather than a soundness deficit (Section 16). On a decision-focused learning task, differentiating through the solver achieves 36% regret reduction versus two-stage predict-then-optimize. The algebraic coefficient extraction path provides 17–226$\times$ speedups for LP and QP subproblems. discopt is available under the EPL-2.0 license, and POUNCE is available as a separate project (https://github.com/jkitchin/pounce) under the EPL-2.0 license.

2 Introduction

Mixed-integer nonlinear programming (MINLP) is a powerful framework for optimization problems involving both discrete choices and nonlinear relationships, with applications spanning process synthesis, network design, scheduling, and machine learning pipeline optimization.[3,4] The dominant algorithmic paradigm for deterministic global MINLP is spatial branch and bound (sBB), which enumerates a search tree of progressively tighter variable-bound regions, solving a convex relaxation at each node.[5,6,7]

State-of-the-art MINLP solvers—BARON,[1] Couenne, SCIP,[2] and Bonmin[8] are monolithic C, C++, or Fortran codebases that have accumulated decades of numerical hardening. While effective, this architecture creates three gaps in the current solver ecosystem.

Gap 1: No differentiable MINLP. Recent work on differentiable optimization has produced layers for quadratic programs (OptNet[9]), convex programs (cvxpylayers[10]), and decision-focused learning for combinatorial problems.[11,12,13] However, none of these approaches handle nonconvex MINLP. Differentiating through an MINLP solver would enable end-to-end learning of cost parameters from decision outcomes—a capability with broad applications in process optimization and operations research.

Gap 2: No batched or GPU-accelerated branch and bound. In spatial B&B, each node requires solving an NLP relaxation. Modern accelerators (GPUs, TPUs) excel at batched computation, yet existing MINLP solvers process nodes sequentially with sparse linear algebra tuned for single-instance performance. A solver built on JAX[14] could use `jax.vmap` to solve batches of node relaxations simultaneously.

Gap 3: No memory-safe, dependency-free IPM. Ipopt[15] is the workhorse NLP solver used inside most MINLP solvers. It depends on Fortran BLAS/LAPACK, the MUMPS sparse direct solver, and C++ infrastructure totaling hundreds of thousands of lines of code. Installing cyipopt on a fresh system requires compiling or locating these Fortran dependencies. A Rust[16] reimplementation would provide memory safety guarantees, zero C/Fortran dependencies, and a clean `cargo build` workflow.

This paper describes discopt and its integration with POUNCE, which together address these three gaps. The contributions are:

1. **Differentiable MINLP** via two mechanisms: the envelope theorem (Level 1) for fast objective sensitivity, and implicit differentiation of the KKT system (Level 3) for full

primal–dual sensitivity matrices, enabling decision-focused learning through nonconvex solvers.

2. **Batched node evaluation** using `jax.vmap` over NLP relaxation solves at B&B nodes, with warm-starting from parent solutions.

3. **Pure-JAX interior point method** with Mehrotra predictor-corrector,[17] Cholesky-based inertia correction, Ipopt-style filter line search,[15] and carry-based `jax.lax.while_loop` for JIT cache reuse.

4. **POUNCE integration**: discopt uses POUNCE, a separate companion project that is a source-level C++ → Rust translation of Ipopt's core IPM algorithm, producing a standalone memory-safe NLP solver with a clean `NlpProblem` trait interface, as one of its NLP backends.

5. **Compositional relaxation framework** driven by a single uniform factorable-relaxation pass over the expression DAG, with McCormick envelopes,[18,19] exact composite convex atoms (log-sum-exp, entropy, norm), α BB underestimators,[20,21] piecewise McCormick with adaptive partitioning[22] achieving 91–94% gap reduction, ICNN-learned relaxations,[23] a sound DCP/SUSPECT-style structural convexity certifier[24,25] that routes certified-convex models to a single-solve fast path, and a generalised-(G-)convexity layer that extends the sound-verdict fast path beyond x-space convexity to log-convex and reparameterisation-convex structure.

6. **Neural network embedding** that formulates trained feedforward networks (ReLU, sigmoid, tanh, softplus) as MINLP constraints via big-M, full-space, and reduced-space strategies, with interval arithmetic bound propagation and ONNX model import.[26,27]

7. **Adaptive Multivariate Partitioning**,[28,29] a complementary global MINLP solver that iterates piecewise-McCormick MILP relaxations with adaptive partition refinement, with the soundness guarantee $LB_k \leq f^* \leq UB_k$ at every iteration.

8. **Model-based design of experiments** under `discopt.doe`: D / A / E / ME-optimal designs on a shared `Experiment` interface, batch / parallel MBDoE,[30,31] identifiability and estimability diagnostics including profile likelihood[32] and the Yao orthogonalisation[33] / Brun collinearity index,[34] and a five-criterion model-discrimination loop covering Hunter–Reiner,[35] Buzzi-Ferraris–Forzatti,[36] Jensen–R{\'e}nyi,[37] Lindley mutual information,[38,39] and DT-compound[40] designs.

9. **Multi-objective optimisation** under `discopt.mo`: scalarisation-based Pareto front tracing via weighted sum, AUGMECON2 ε-constraint,[41] weighted Tchebycheff, NBI,[42] and NNC,[43] with hypervolume / IGD / spread / ε-indicator quality metrics[44,45] and full inheritance of the MINLP / differentiable / global-solver stack on every scalarised subproblem.

10. **Cutting-plane framework** with reformulation-linearization (RLT),[46] second-order-cone, and moment/PSD eigenvalue separators,[47] governed by a structure-gated `cuts`'auto'= policy that selects the separator family from the detected problem structure.

11. **Complementarity constraints (MPEC)** under `discopt.mpec`: a `complementarity(f, g)` declaration with three exact reformulations — Scholtes regularisation,[48] SOS1,[49] and generalized-disjunctive lowering[50] — each reducing to a standard discopt `Model`.

12. **Automatic geometric-program detection** under `discopt.gp`: a sound log-convexity verdict[51,52] that recognises posynomial programs and routes them through a single log-space convex solve, extended to *GP-structured MINLP* (integer-augmented posynomial models solved by integer branch-and-bound over log-space convex node relaxations).

13. **Differentiable mixed-integer programming** via fix-and-differentiate, and **infeasibility analysis** (irreducible infeasible subsystems[53] and conflict-driven no-good cuts[54]).

14. **Optimisation under uncertainty**: a stochastic-programming module under `discopt.stochastic` (two-stage recourse and extensive form, multicut L-shaped / Benders decomposition,[55] progressive hedging,[56] sample-average approximation with Mak–Morton–Wood bounds,[57] and CVaR / risk-averse objectives[58,59]) and a bilevel-optimisation module under `discopt.bilevel` that reduces a Stackelberg leader–follower program with a convex lower level to a single-level MPEC via KKT or strong-duality reformulation.[60]

15. **A certificate-soundness audit.** Because a global solver's product is its *certificate*, we report a systematic soundness audit — roughly three dozen correctness issues (all fixed) plus a twenty-module code review — that hardened the certificate across relaxation math, presolve/FBBT, branch-and-bound status logic, cuts, the `.nl` parser, and the LP/dual layer, maintaining =incorrect_count = 0= on the certifying panel (Section 15).

The remainder of this paper is organised as follows. Section 4 describes the discopt architecture, including the modelling API, DAE collocation, neural network embedding, and compiler infrastructure. Section 5 presents the three NLP solver backends. Section 7 details the convex relaxation framework, including the structural convexity certifier of Section 7.6. Section 8 introduces the AMP global solver, and Section 9 the branch-and-bound search enhancements (primal heuristics, node selection, conflict cuts, and infeasibility analysis). Section 10 covers differentiable optimisation, including differentiable mixed-integer programs. Section 11 discusses dynamic optimisation, Section 12 parameter estimation, Section 13 model-based design of experiments (with identifiability / estimability / discrimination diagnostics), and Section 14 multi-objective optimisation. Section 15 records the certificate-soundness audit. Section 16 reports computational experiments. The final two sections discuss limitations and future work, and conclude.

3 Background and Related Work

3.1 Spatial Branch and Bound for MINLP

Spatial branch and bound (sBB) is the dominant paradigm for deterministic global optimization of MINLP problems.[3] The algorithm maintains a search tree where each node represents a subproblem defined by tightened variable bounds. At each node, a convex relaxation of the

original nonconvex problem is solved; the relaxation's optimal value provides a valid lower bound on the global optimum within that region.

The framework was introduced by Land and Doig[5] for integer programming and extended by Dakin.[6] Smith and Pantelides[7] formalized spatial B&B for nonconvex MINLP by combining variable-bound branching with factorable relaxation construction.

Modern sBB solvers enhance the basic framework with bound tightening techniques—feasibility-based bound tightening (FBBT) and optimization-based bound tightening (OBBT)[61,62]—which propagate bound improvements through the expression graph, and primal heuristics such as the feasibility pump[63] that attempt to find good incumbent solutions early in the search.

3.2 Convex Relaxations

The quality of the convex relaxation at each node directly determines the effectiveness of the B&B search. Weaker relaxations produce loose lower bounds, leading to more nodes explored before convergence.

McCormick relaxations[18] construct convex and concave envelopes for factorable functions by composing relaxations of elementary operations (addition, multiplication, univariate functions). Tsoukalas and Mitsos[19] extended this to the multivariate case. The α BB method[20,21] adds a separable quadratic perturbation to ensure convexity, where the perturbation magnitude α is determined by the minimum eigenvalue (or Gershgorin estimate) of the objective Hessian.

Piecewise McCormick relaxations[22] partition the variable domain into subintervals and construct tighter McCormick envelopes on each partition, significantly reducing the relaxation gap at the cost of additional auxiliary variables. Outer approximation (OA)[64,65] and reformulation-linearization technique (RLT)[46] provide complementary cutting-plane approaches.

3.3 Interior Point Methods for NLP

The continuous NLP relaxation at each B&B node is typically solved by an interior point method (IPM). The primal-dual IPM[66,67] transforms inequality constraints into barrier terms in the objective, then applies Newton's method to the resulting KKT system. Key algorithmic components include the Mehrotra predictor-corrector step,[17] which adaptively adjusts the centering parameter for faster convergence, and filter line search,[68,15] which accepts steps that improve either the objective or constraint feasibility.

Ipopt[15] is the most widely used open-source NLP solver, implementing a filter line-search interior point method with inertia correction, feasibility restoration, and warm-starting. It has accumulated over 20 years of numerical hardening for edge cases including degenerate Jacobians, rank-deficient KKT systems, and cycling detection.

Sensitivity analysis of NLP solutions with respect to problem parameters is well-established.[69] Pirnay et al.[70] implemented the sIPOPT framework within Ipopt for efficient parametric sensitivity computation by reusing the KKT factorization.

3.4 Differentiable Optimization: Prior Work

The idea of differentiating through an optimization solver to enable end-to-end learning has received significant recent attention. Amos and Kolter[9] introduced OptNet for differentiating through quadratic programs. Agrawal et al.[10] extended this to general convex programs via cvxpylayers. Elmachtoub and Grigas[11] formalized the "smart predict, then optimize" framework, and Wilder et al.[12] and Mandi et al.[13] developed methods for combinatorial optimization with decision-focused learning.

A critical gap remains: **none of these approaches handle nonconvex MINLP**. Existing differentiable optimization layers require convexity (OptNet, cvxpylayers) or use surrogate loss functions that approximate but do not exactly differentiate through the solver (decision-focused learning for combinatorics). discopt fills this gap by providing exact gradients through a nonconvex NLP solver via implicit differentiation of the KKT conditions.

3.5 Gaps in the State of the Art

Table 1 summarizes the capabilities of existing systems versus discopt.

Feature	BARON	SCIP	Bonmin	cvxpylayers	OMLT	discopt
Nonconvex MINLP	Yes	Yes	Yes	No	Via Pyomo	Yes
Differentiable solve	No	No	No	Yes (convex)	No	Yes
Batched node evaluation	No	No	No	N/A	No	Yes
Memory-safe solver	No	No	No	N/A	No	Yes
Zero C/Fortran deps	No	No	No	Yes	No	Yes
DFL through MINLP	No	No	No	No	No	Yes
Compositional relaxation	Yes	Partial	No	N/A	No	Yes
NN embedding in MINLP	No	No	No	No	Yes	Yes
Robust counterpart API	No	No	No	No	No	Yes
Adjustable RO (ADR)	No	No	No	No	No	Yes

Table 1: Comparison of discopt with existing optimization frameworks. DFL = decision-focused learning.

4 The discopt Architecture

4.1 System Overview

discopt is organized as a three-layer system: (1) a Python modeling API for problem formulation, (2) a JAX-based numerical layer for evaluation, automatic differentiation, and batch computation, and (3) a Rust backend for tree management, expression structure detection, and presolve.

```
1  User Model (Python)
2       |
```

```
      v
Expression DAG ---> DAG Compiler ---> JAX Functions (JIT/grad/vmap)
      |                                       |
      v                                       v
Rust Expression IR             NLP Evaluator (obj, grad, Hess, Jac)
      |                                       |
      v                                       v
B&B Tree Manager               NLP Solver Backend
(node pool, branching,         (IPM / POUNCE / Ipopt)
 bound tightening)                     |
      |                                v
      +-------- batch dispatch --------+
                     |
                     v
              SolveResult
```

An automatic problem classifier inspects the expression DAG to categorize problems as LP, QP, NLP, MILP, MIQP, or MINLP. For LP and QP subclasses, discopt dispatches to specialized IPM solvers that extract algebraic coefficients directly from the DAG, bypassing JAX tracing and autodiff entirely. This algebraic extraction path provides 100–226× speedups over the generic NLP path (\S 7.4).

Core and plugins. To keep the core lean, the domain-specific *application builders* — model-based design of experiments (`discopt.doe`), the teaching / tutor front-ends, and the alternating-current optimal-power-flow and pooling builders (`discopt.opf`, `discopt.pooling`) — have been extracted into standalone `discopt-doe`, `discopt-course`, and `discopt-apps` distributions. Because `discopt` is a namespace package, installing a plugin restores its `discopt.<name>` import path unchanged, and each plugin registers its command-line front-end through a `discopt.cli` entry-point hook so that `discopt <subcommand>` dispatches to it. A stable public `discopt.parametric` API compiles model expressions into JAX-differentiable functions of (x, p) without reaching into internal modules, giving both in-tree consumers (parameter estimation) and external plugins (DoE, microkinetics) a single supported contract. The core retains the general MINLP machinery — GDP, DAE collocation, neural-network embedding, robust and stochastic optimisation — while domain builders live as plugins.

4.2 Modeling API

The modeling API uses operator overloading to construct an expression DAG that maps to both JAX-traceable functions and Rust expression IR:

```python
import discopt.modeling as dm

m = dm.Model("portfolio")
x = m.continuous("weights", shape=(n,), lb=0, ub=1)
y = m.binary("selected", shape=(n,))

m.minimize(risk @ (x * x) - expected_return @ x)
m.subject_to(dm.sum(x) == 1, name="budget")
m.subject_to(x <= y, name="indicator")  # x_i > 0 => y_i = 1
m.subject_to(dm.sum(y) <= k, name="cardinality")

result = m.solve()
```

The API supports continuous, binary, and integer variables; parameters (for sensitivity analysis and decision-focused learning); generalized disjunctive programming (GDP) constraints with big-M, multiple big-M (MBM) with LP-based bound tightening, convex hull, and Logic-based Outer Approximation (LOA[71]) reformulations via `gdp_method`; `BooleanVar` with propositional logic operators (`&`, `|`, `~`, `.implies()`, `.equivalent_to()`) and CNF-to-linear conversion; nested disjunctions for hierarchical process design; and standard mathematical functions (`exp`, `log`, `sin`, `cos`, `sqrt`, `abs`, `tanh`, `sigmoid`, `softplus`, and matrix operations).

4.3 Dynamic Optimization via DAE Collocation

The `discopt.dae` module extends the modeling API to dynamic optimization problems by transcribing ODE/DAE systems into algebraic NLP constraints using orthogonal collocation on finite elements.[72,73] Users declare state, algebraic, and control variables, supply an ODE/DAE right-hand side as a Python callable that builds discopt expressions, and call `discretize()` to generate all collocation and continuity constraints automatically. Radau IIA and Gauss-Legendre schemes with 1–5 points per element, second-order ODEs via state augmentation, index-1 DAEs, piecewise-constant controls, and integral objectives are all supported. The transcription is covered in detail in the Dynamic Optimization section (11); parameter estimation and model-based design of experiments for dynamic systems are built on the same machinery (12, 13).

4.4 Neural Network Embedding

The `discopt.nn` module embeds trained feedforward neural networks as algebraic constraints in MINLP models, enabling optimization over machine learning surrogates with global optimality guarantees. This follows the approach of OMLT,[26] adapted to discopt's expression DAG, JAX automatic differentiation, and McCormick relaxation framework.

An `NNFormulation` builder (following the same pattern as `DAEBuilder`) takes an existing `Model`, a `NetworkDefinition` describing the trained network's weights, biases, and activation functions, and a formulation strategy, then adds all necessary variables and constraints:

```
from discopt.nn import NNFormulation, NetworkDefinition, DenseLayer, Activation

net = NetworkDefinition([
    DenseLayer(W1, b1, Activation.RELU),
    DenseLayer(W2, b2, Activation.LINEAR),
], input_bounds=(lb, ub))

nn = NNFormulation(model, net, strategy="relu_bigm")
nn.formulate()
model.minimize(dm.sum(nn.outputs))
```

Three formulation strategies are provided:

- **Full-space**: Explicit pre-activation ($\hat{z}$) and post-activation (z) variables for each neuron, with affine constraints $\hat{z} = Wx + b$ and activation constraints $z = \sigma(\hat{z})$. Supports smooth activations (sigmoid, tanh, softplus) that integrate directly with JAX autodiff and McCormick relaxations for spatial B&B.

- **ReLU big-M**.[27] For networks with ReLU activations, introduces binary variables q_j and linear big-M constraints: $z \geq 0$, $z \geq \hat{z}$, $z \leq u \cdot q$, $z \leq \hat{z} - l(1-q)$, where l, u are pre-activation bounds from interval arithmetic. Neurons with bounds that are always positive (or always negative) are detected and simplified to avoid unnecessary binary variables.

- **Reduced-space**: Builds a single nested expression with no intermediate variables, suitable for small networks.

Interval arithmetic bound propagation through the network layers computes tight big-M constants by splitting weight matrices into positive and negative parts. An ONNX reader (`load_onnx`) imports trained models from scikit-learn, PyTorch, and Keras via the ONNX interchange format. Beyond feedforward networks, the module also embeds decision trees and tree ensembles via a per-leaf MILP encoding.

The soundness audit of Section 15 closed two embedding-specific hazards. For a network embedded behind an affine input/output scaling, bound propagation now runs over the *scaled* box the layers actually consume, so the ReLU big-M constants are a valid over-estimate rather than being computed from the wrong (unscaled) domain; and the per-leaf big-M in the tree-ensemble encoding is now a per-constraint constant that stays inert for a non-selected leaf even when a split threshold falls outside the variable box, so an out-of-box threshold no longer cuts feasible points. On the reader side, the ONNX path now applies `Gemm` scaling attributes, consumes residual `Add` biases correctly, and rejects non-sequential (branched) graphs instead of silently flattening them, while the scikit-learn reader respects the output activation of an MLP classifier and the base score of a gradient-boosted ensemble.

4.5 Robust Optimization Module

The `discopt.ro` module converts a nominal model with uncertain parameters into a deterministic **robust counterpart** that is feasible for every realization of the uncertainty within a prescribed set. Following the classical framework of Ben-Tal and Nemirovski,[74,75] robust optimization replaces the original problem

```
min_x  f(x, p)      s.t. g_i(x, p) <= 0  for all p in U
```

with a deterministic equivalent that encodes the worst-case constraint tightening analytically.

4.5.1 Uncertainty Sets

Three standard uncertainty set families are supported:

- **Box** (L_∞ ball) $\mathcal{U} = \{p : |p_j - \bar{p}_j| \leq \delta_j\}$: the worst-case maximisation of an affine function $c^\top p$ over the box is $\bar{c}^\top p + \sum_j \delta_j |c_j|$, yielding a separable 1-norm penalty. When p appears only as an additive RHS term, this collapses to a single bound substitution $p \leftarrow \bar{p} \pm \delta$.

- **Ellipsoidal** $\mathcal{U} = \{p : \|\Sigma^{-1/2}(p - \bar{p})\|_2 \leq \rho\}$: the worst-case is $\bar{a}^\top p + \rho\|\Sigma^{1/2}a(x)\|_2$, a second-order cone (SOC) penalty.[74]
- **Polyhedral** $\mathcal{U} = \{p : A\xi \leq b,\ \xi = p - \bar{p}\}$: the support function is evaluated by LP duality, introducing auxiliary dual variables $\lambda \geq 0$ satisfying $A^\top\lambda = a(x)$ and adding $b^\top\lambda$ to the constraint.[76]

The **budget-of-uncertainty** set (Bertsimas–Sim[76]) is provided as a convenience constructor:

$$\mathcal{U}_\Gamma = \left\{ p : |p_j - \bar{p}_j| \leq \delta_j,\ \sum_j \frac{|p_j - \bar{p}_j|}{\delta_j} \leq \Gamma \right\}$$

The budget $\Gamma \in [0, k]$ interpolates between the nominal solution ($\Gamma = 0$) and full box robustness ($\Gamma = k$), controlling the trade-off between solution cost and protection level.

4.5.2 Implementation: Reformulation Strategies

The box reformulation uses two strategies depending on the structure of the expression DAG. When uncertain parameters appear with constant (known-sign) coefficients, the reformulation traverses the DAG while tracking the **effective sign** of each uncertain parameter at each node. At a subtraction node $a - p$, the parameter p acquires sign -1; at a negation node $-p$, the sign flips. Multiplication by a non-negative constant preserves the sign; multiplication by a negative constant flips it. This sign determines whether the worst-case value is the upper or lower component bound:

```
sign > 0 and maximize  =>  use upper bound  (pbar + delta)
sign < 0 and maximize  =>  use lower bound  (pbar - delta)
```

When the expression contains **bilinear** terms ($p \cdot x$ where both are free), as arises from affine decision rules, the coefficient of p involves decision variables whose sign is not known a priori. In this case the reformulation extracts the per-component coefficient of each uncertain parameter and applies absolute-value linearization: the worst-case of $\text{coeff}(x) \cdot \xi$ over $|\xi| \leq \delta$ is $\delta \cdot |\text{coeff}(x)|$, where the absolute value is linearized via auxiliary variables $t \geq 0$ satisfying $\text{coeff}(x) \leq t$ and $-\text{coeff}(x) \leq t$. The implementation automatically detects the presence of bilinear terms and selects the appropriate strategy.

For the polyhedral uncertainty set, the reformulation introduces auxiliary dual variables $\lambda \geq 0$ (one per polytope row) with equality constraints $A^\top\lambda = \text{coeff}(x)$ and a worst-case penalty $b^\top\lambda$. This LP-duality approach correctly handles the budget-of-uncertainty constraint, where the budget Γ couples the uncertainty components and component-wise bounds alone would be insufficient.

The `RobustCounterpart` class follows the same builder pattern as `NNFormulation`:

```python
from discopt.ro import BoxUncertaintySet, RobustCounterpart

m = dm.Model("production")
x = m.continuous("x", shape=(3,), lb=0)
c = m.parameter("c", value=[10.0, 15.0, 8.0])
```

```
d = m.parameter("d", value=100.0)

m.minimize(dm.sum(c * x))
m.subject_to(dm.sum(x) >= d, name="demand")

rc = RobustCounterpart(m, [
    BoxUncertaintySet(c, delta=0.10 * c.value),   # costs ś 10%
    BoxUncertaintySet(d, delta=0.05 * d.value),   # demand ś 5%
])
rc.formulate()          # modifies m in-place; m is now deterministic
result = m.solve()
```

After `formulate()`, the model contains no uncertain parameters: each has been replaced by its worst-case constant. The reformulated model is a standard MINLP that can be solved by any of discopt's solver backends.

4.5.3 Correctness Guarantee

For the additive/affine uncertainty case, both reformulation strategies produce an exact robust counterpart: the reformulated constraints hold for all $p \in \mathcal{U}$ if and only if they hold after substitution. The sign-tracking path follows directly from LP duality;[74] the coefficient-extraction path is exact because the absolute-value linearization is tight for box uncertainty. The polyhedral reformulation via LP duality is exact for any polyhedral set, including the budget-of-uncertainty case.[76] The ellipsoidal reformulation produces exact second-order cone constraints for affine uncertainty. The implementation verifies correctness via structural tests that confirm (1) no uncertain parameters remain in the reformulated DAG, and (2) the reformulated model solves to the analytically derived robust optimal value.

The soundness audit of Section 15 sharpened this guarantee in three places. First, a bare parameter–variable product $p \cdot x$ with a sign-*indefinite* variable factor is now routed through the absolute-value linearization rather than the sign-tracking fast path (which is sound only when the variable is provably non-negative), closing an under-protection hole. Second, `RobustCounterpart.formulate()` now asserts that *no* uncertain parameter survives robustification, raising rather than silently returning a non-robust model when a pattern is unsupported. Third, the budget-of-uncertainty constructor was corrected to encode the *exact* Bertsimas–Sim set ? the earlier implementation stored only the all-plus/all-minus budget facets and so protected against a strict superset of the true set — sound (over-protective by up to a factor of two in mixed-sign directions) but more conservative than the promised set. The corrected counterpart stores the compact lifted (ξ, u) representation, which the polyhedral LP-duality path dualises exactly; it was verified robust by exhaustive in-set sampling (worst in-set constraint slack within tolerance across all vertices, a dense grid, and interior samples) while recovering the less-conservative Bertsimas–Sim optimum.

4.5.4 Adjustable Robust Optimization via Affine Decision Rules

Static robust optimization requires all decisions to be fixed before the uncertain parameters are revealed. When some decisions can be deferred until after (partial) observation of the uncertainty, adjustable robust optimization (ARO)[77] provides a less conservative solution.

The two-stage framework partitions variables into **here-and-now** decisions x (chosen before observing ξ) and **wait-and-see** recourse variables $y(\xi)$ (chosen after). Optimising

over arbitrary functions $y(\cdot)$ is generally intractable (the problem is semi-infinite), so discopt adopts *affine decision rules* (also called linear decision rules).[77,78]

$$y(\xi) = y_0 + \sum_{j=1}^{n_\xi} Y_j \cdot \xi_j, \quad \xi_j = p_j - \bar{p}_j$$

where y_0 and $\{Y_j\}$ are new ordinary decision variables representing the intercept and policy matrix columns. Bertsimas and Goyal[79] establish that for LP problems with right-hand-side uncertainty, affine rules are **optimal**, meaning no other measurable recourse policy achieves a lower cost; for uncertain constraint matrices they are an inner approximation.

The `AffineDecisionRule` class implements this substitution as a pre-processing step compatible with the existing `RobustCounterpart`:

```
from discopt.ro import AffineDecisionRule, BoxUncertaintySet, RobustCounterpart

# Stage 1: substitute recourse variable y with y0 + Y0 * (d - dbar)
adr = AffineDecisionRule(y, uncertain_params=d)
adr.apply()          # retires y; adds y0, Y0 to model._variables

# Stage 2: static robust counterpart handles remaining xi terms
rc = RobustCounterpart(m, BoxUncertaintySet(d, delta=20.0))
rc.formulate()       # fully deterministic; optimise over (x, y0, Y0)
```

The DAG substitution walks every constraint and objective node, replacing Variable nodes that match the recourse variable with the affine expression via identity comparison ($O(|V| + |E|)$ in the DAG size). After substitution the recourse variable is retired from the model's variable list and remaining variables are renumbered. In addition, `apply()` automatically enforces robust feasibility of the recourse variable bounds: if the original variable y had bounds $[l, u]$, the constraints $y_0 + \sum_j Y_j \xi_j \leq u$ and $y_0 + \sum_j Y_j \xi_j \geq l$ are added to the model, ensuring that the affine rule respects the original domain for all realizations of ξ. The subsequent `RobustCounterpart.formulate()` call handles the bilinear $Y_j \cdot \xi_j$ terms in these bound constraints via coefficient extraction and absolute-value linearization.

A call to `AffineDecisionRule.apply()` followed by `RobustCounterpart.formulate()` leaves the model fully deterministic with (1) no recourse variables, (2) no uncertain parameters, and (3) new intercept and policy-column variables in their place. For simple problems (e.g., a two-variable LP with a single uncertain parameter), the affine decision rule typically matches the static robust optimal value, because the static solution already adapts optimally. The theoretical advantage of ADR emerges in problems with structural asymmetry: multiple constraints, integer first-stage variables, or high-dimensional uncertainty where the static worst case is overly conservative.

4.6 Complementarity Constraints and MPEC

Mathematical programs with equilibrium (complementarity) constraints (MPECs)[50] arise whenever an inner equilibrium — contact mechanics, Karush–Kuhn–Tucker conditions of a lower-level problem, piecewise-linear device characteristics, or economic complementary slackness — is embedded in an outer optimisation. The defining relation is the complementarity condition $0 \leq f(x) \perp g(x) \geq 0$, requiring $f, g \geq 0$ component-wise together with

$f_i\, g_i = 0$. The product constraint violates every standard constraint qualification at the origin, so a naive nonlinear formulation stalls at spurious stationary points.

The `discopt.mpec` module exposes a single declarative primitive, `complementarity(f, g, name...)`=, and lowers a set of such conditions to a standard discopt `Model` through three exact reformulations selected by `solve_mpec(model, pairs, method...)`=:

- **Scholtes regularisation** (`method`"scholtes"=, the default). The exact product $f_i\, g_i = 0$ is relaxed to $f_i\, g_i \le t$ and a homotopy drives $t \downarrow 0$ through a short sequence of local NLP solves.[48] Scholtes established that limit points of this scheme are C-stationary, and B-stationary under mild conditions, making it the method of choice when a fast locally-optimal MPEC solution suffices.
- **SOS1** (`method`"sos1"=). The pair (f_i, g_i) is placed in a special-ordered-set-of-type-1,[49] encoding "at most one of f_i, g_i is nonzero" exactly; branch and bound then resolves the discrete choice, yielding a globally valid solution.
- **Disjunctive lowering** (`method`"gdp"=). The condition is expressed as the disjunction $(f_i = 0) \vee (g_i = 0)$ and handed to the existing GDP machinery (Section 4), which applies big-M or convex-hull reformulation with the same bound-tightening used elsewhere.

Because all three reformulations emit an ordinary `Model`, an MPEC inherits the full solver stack: the regularised path uses any NLP backend, while the SOS1 and disjunctive paths are solved to global optimality by spatial branch and bound with the relaxations of Section 7. Nonlinear complementarity operands f, g are lifted to auxiliary variables so the disjunctive and big-M encodings remain exact on bilinear and higher-degree expressions.

4.7 Stochastic Programming

Where the robust-optimisation module of Section 4 protects against *every* realisation in an uncertainty set, the `discopt.stochastic` module takes the complementary, distributional view: a two-stage program with recourse minimises a first-stage *here-and-now* cost plus the expected (or risk-adjusted) cost of a second-stage *wait-and-see* recourse taken once the uncertainty is realised.[59] It is the distributional sibling of `discopt.ro`.

Uncertainty is carried by a `ScenarioSet` — a finite, probability-weighted collection of `Scenario` objects, built explicitly, from a list, or by Monte-Carlo sampling of user-supplied per-parameter samplers (`ScenarioSet.sample`). `build_extensive_form` assembles the deterministic-equivalent *extensive form*: given a first-stage cost and a `recourse_builder(model, scenario_data, s)` callback that stamps the per-scenario recourse constraints, it forms the scalarised objective $c^\top x + \sum_s p_s\, Q_s(x)$ and hands back an ordinary `Model` that the full MINLP stack then solves.

The risk functional is pluggable through a `RiskMeasure` interface. `Expectation` gives the risk-neutral $\sum_s p_s Q_s$; `CVaR($\alpha$)` adds the Rockafellar–Uryasev conditional value-at-risk[58] via its convex epigraph reformulation; `MeanCVaR($\lambda,\alpha$)` interpolates between the two; and `chance_constraint` enforces a sample-average probabilistic constraint $\mathbb{P}(g(x, \xi) \le 0) \ge 1 - \varepsilon$ through per-scenario big-M indicators.

Two decomposition solvers avoid materialising the full extensive form on large scenario trees. `solve_lshaped` runs a probability-weighted multicut L-shaped (Benders) decomposition,[55] dispatching between classical L-shaped LP recourse and generalized Benders for convex-nonlinear recourse. `progressive_hedging` implements the Rockafellar–Wets scenario-decomposition scheme,[56] driving per-scenario first-stage copies to a common non-anticipative value through an augmented-Lagrangian penalty. For solution-quality certification, `saa_lower_bound_estimate` / `saa_upper_bound_estimate` form the Mak–Morton–Wood statistical bounds that bracket the true optimum from independent sample batches,[57] and `optimality_gap_estimate` reports the resulting gap. A distributionally-robust primitive (`worst_case_expected_cost`) evaluates the worst-case expectation over a total-variation (L_1) ambiguity ball around the nominal scenario probabilities; the full single-level DRO reformulation and multistage (nested-Benders / SDDP) recourse are reserved for future work.

4.8 Bilevel Optimization

A bilevel (Stackelberg) program optimises a *leader's* objective subject to a *follower* that itself solves an optimisation problem parameterised by the leader's decision.[60] When the lower level is convex in the follower variables (an LP or convex QP), it can be replaced by its Karush–Kuhn–Tucker conditions, collapsing the two-level problem into a single-level mathematical program with equilibrium constraints (Section 4.6). The `discopt.bilevel` module exposes a `BilevelProblem` builder that takes the leader model together with the follower's variables, objective, and constraints, and rewrites it in place under `formulate()`:

- **KKT reformulation** (`method"kkt"=`). The follower is replaced by its stationarity $\nabla_y L = 0$ (emitted symbolically over the expression DAG so the resulting constraints stay on the certifiable global path), primal and dual feasibility, and per-row complementarity $0 \le \mu_i \perp -g_i \ge 0$ handed to `discopt.mpec` under a selectable `mpec_method` (`gdp` or `sos1`).
- **Strong-duality reformulation** (`method"strong_duality"=`). The same stationarity and feasibility conditions are combined with a single aggregate strong-duality equality in place of per-row complementarity, avoiding disjunctive branching.

The builder handles the *optimistic* bilevel formulation and refuses loudly — rather than silently returning a wrong answer — on cases outside its soundness envelope: integer follower variables, a lower level nonconvex in y, or pessimistic semantics. Because both reformulations emit an ordinary `Model`, a bilevel program inherits the same spatial branch and bound, relaxations, and NLP backends as the rest of discopt.

4.9 JAX DAG Compiler

The DAG compiler walks the expression tree and produces pure JAX functions compatible with `jax.jit`, `jax.grad`, and `jax.vmap`. Each expression node type (`Variable`, `Constant`, `BinaryOp`, `FunctionCall`, etc.) maps to a corresponding JAX operation. The compiler handles:

- **Lazy evaluation**: expressions are not evaluated until the DAG is compiled
- **Automatic differentiation**: the JAX function graph supports forward and reverse mode AD for gradients, Hessians, and Jacobians
- **Vectorization**: `jax.vmap` can map the compiled function over batches of variable values or bound vectors
- **JIT compilation**: compiled functions are traced and optimized by XLA for efficient repeated evaluation

4.10 Rust Backend

The Rust backend (`discopt-core` and `discopt-python` crates) provides three components:

Expression IR. A typed expression intermediate representation in Rust that mirrors the Python DAG but enables structure detection on the Rust side. PyO3 bindings[80] provide zero-copy data transfer between Python `numpy` arrays and Rust `Vec<f64>` via `into_pyarray()`.

Branch-and-bound tree. The `PyTreeManager` manages the B&B node pool in Rust with configurable selection strategies (best-first, depth-first). Node bounds, solutions, and parent solution vectors are stored in Rust memory. The Python layer calls `export_batch(k)` to retrieve k open nodes, solves the NLP relaxations (in Python/JAX), and returns results via `import_results()`. This separation keeps the tree manipulation in Rust (zero GIL contention) while leveraging JAX for numerical computation.

Presolve. Feasibility-based bound tightening (FBBT) and probing propagate domain reductions through the expression graph. A `.nl` file parser enables importing problems from the MINLPLib benchmark library.[81]

5 NLP Solver Backends

discopt supports three interchangeable NLP solver backends, selectable via `Model.solve(nlp_solver...)`=. All three implement the same interface: given an objective function $f(x)$, constraints $g(x)$, variable bounds $x_l \leq x \leq x_u$, and constraint bounds $g_l \leq g(x) \leq g_u$, find x^* that minimizes f subject to all constraints.

5.1 Pure-JAX Interior Point Method

The pure-JAX IPM implements a primal-dual barrier method that is fully JIT-compilable and vmappable. The algorithm solves the augmented KKT system:

$$\begin{bmatrix} H + \Sigma + \delta_w I & J^T \\ J & -D - \delta_c I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} r_x \\ r_y \end{bmatrix} \tag{1}$$

where $H = \nabla^2_{xx}\mathcal{L}$ is the Hessian of the Lagrangian, $\Sigma = \text{diag}(z_l/s_l + z_u/s_u)$ is the barrier Hessian diagonal, J is the constraint Jacobian, D is the condensed slack diagonal for inequality constraints, and δ_w, δ_c are inertia correction regularizations.

5.1.1 Mehrotra Predictor-Corrector

Following Mehrotra,[17] each iteration consists of two linear system solves:

1. **Affine predictor** ($\mu = 0$): solve the KKT system with zero centering to obtain $(\Delta x^{\text{aff}}, \Delta y^{\text{aff}})$. Compute the affine complementarity μ^{aff}.
2. **Centering parameter**: $\sigma = (\mu^{\text{aff}}/\mu^{\text{curr}})^3$, clipped to $[0, 1]$.
3. **Corrector**: re-solve with centering $\sigma\mu$ and cross-product correction terms $\Delta s_i \Delta z_i / s_i$.

The corrector step provides second-order information about the central path, typically reducing the iteration count by 30–50\% compared to a single Newton step.

5.1.2 Inertia Correction

The KKT system must have correct inertia (n positive, m negative eigenvalues) for the Newton direction to be a descent direction. We detect incorrect inertia via Cholesky factorization: if the upper-left block $W = H + \Sigma + \delta_w I$ produces NaN in its Cholesky factor, the inertia is incorrect. The regularization δ_w is increased by a factor of 8 (up to $\delta_w^{\max} = 10^{40}$) until correct inertia is achieved. This Cholesky-based detection is approximately $3\times$ faster than eigenvalue decomposition.

5.1.3 Filter Line Search

Step acceptance follows Ipopt's filter line search.[15] The algorithm maintains an ℓ_1 merit function $\phi(x) = f(x) + \nu\|c(x)\|_1$ with adaptive penalty parameter ν and performs backtracking with Armijo-type sufficient decrease. A stall detection mechanism triggers projected steepest-descent recovery after 3 consecutive line search failures, handling the Maratos effect near active bound constraints.

5.1.4 Carry-Based `while_loop`

A critical implementation detail for JAX performance: all problem data (bounds, constraint bounds, masks) is placed in the `while_loop` carry state as a `NamedTuple`, rather than captured in the closure. Closure-captured data forces JIT retracing for each new problem instance, while carry-based data allows JIT cache reuse. Problem dimensions are derived from array shapes (`c.shape[0]`, `b.shape[0]`) rather than stored as integers, since integers in the carry become JAX tracers.

5.1.5 Batch Solving via `jax.vmap`

The `solve_nlp_batch` function applies `jax.vmap` over the initial point and variable bound dimensions:

```
def solve_nlp_batch(obj_fn, con_fn, x0_batch, xl_batch, xu_batch, g_l, g_u, opts):
    def _solve_single(x0, xl, xu):
        return ipm_solve(obj_fn, con_fn, x0, xl, xu, g_l, g_u, opts)
    return jax.vmap(_solve_single)(x0_batch, xl_batch, xu_batch)
```

This solves all k node relaxations simultaneously, providing approximately k-fold parallelism on hardware with sufficient FLOP throughput. The objective and constraint functions are shared across the batch; only the starting points and variable bounds vary per instance.

At the root node of the B&B tree, multi-start solving uses 5 diverse starting points (midpoint, lower-quarter, upper-quarter, and 2 random) solved simultaneously via `vmap`. On non-root nodes, the solver warm-starts from the parent node's solution clipped to the child's tightened bounds.

5.2 POUNCE: A Memory-Safe IPM in Rust

POUNCE is a source-level translation of Ipopt's core interior point algorithm from C++ to Rust, developed as a separate companion project (https://github.com/jkitchin/pounce). It is **not** a wrapper around C++ Ipopt; it is a standalone Rust crate that produces native code with Rust's ownership and borrowing guarantees. discopt integrates POUNCE as one of its NLP backends through POUNCE's Python bindings (PyPI distribution `pounce-solver`, import name `pounce`).

5.2.1 Motivation

Three considerations drove the decision to translate rather than wrap:

1. **Numerical hardening.** Ipopt handles hundreds of edge cases accumulated over two decades: degenerate Jacobians, rank-deficient KKT systems, cycling detection, and tiny-step recovery. A from-scratch IPM would rediscover these failures one benchmark at a time. Translation preserves this institutional knowledge.

2. **Memory safety.** Rust's ownership model eliminates use-after-free bugs, data races, and buffer overflows at compile time—categories of bugs that have affected C++ numerical codes. The `cargo clippy` linter and optional `miri` interpreter catch additional issues.

3. **Deployment simplicity.** POUNCE compiles with `cargo build` with zero C or Fortran dependencies. The entire solver is a single Rust crate, eliminating the MUMPS/BLAS/LAPACK dependency chain that makes Ipopt installation non-trivial.

5.2.2 Architecture

POUNCE exposes a clean `NlpProblem` trait that users implement:

```rust
pub trait NlpProblem {
    fn num_variables(&self) -> usize;
    fn num_constraints(&self) -> usize;
    fn bounds(&self, x_l: &mut [f64], x_u: &mut [f64]);
    fn constraint_bounds(&self, g_l: &mut [f64], g_u: &mut [f64]);
    fn initial_point(&self, x0: &mut [f64]);
    fn objective(&self, x: &[f64]) -> f64;
    fn gradient(&self, x: &[f64], grad: &mut [f64]);
    fn constraints(&self, x: &[f64], g: &mut [f64]);
    fn jacobian_structure(&self) -> (Vec<usize>, Vec<usize>);
    fn jacobian_values(&self, x: &[f64], vals: &mut [f64]);
```

```
    fn hessian_structure(&self) -> (Vec<usize>, Vec<usize>);
    fn hessian_values(&self, x: &[f64], obj_factor: f64,
                      lambda: &[f64], vals: &mut [f64]);
}
```

The solver implements filter line search with Ipopt-style acceptance criteria, inertia correction via the `LinearSolver` trait (pluggable backends), feasibility restoration, warm-starting from previous solutions, and configurable convergence criteria.

5.2.3 PyO3 Integration

POUNCE ships PyO3 bindings[80] that discopt drives directly. A `PyNlpProblem` struct implements the `NlpProblem` trait by calling back into Python evaluator methods (`evaluate_objective`, `evaluate_gradient`, `evaluate_constraints`, `evaluate_jacobian`, `evaluate_lagrangian_hessian`). Bounds and sparsity structure are cached in Rust-owned memory. The main solve loop releases the Python GIL via `py.allow_threads()`, and callbacks re-acquire it only for the duration of each evaluation:

```
let result = py.allow_threads(|| pounce::solve(&problem, &opts));
```

This means the heavy linear algebra in the IPM iteration runs without GIL contention, while the JAX-compiled gradient and Hessian evaluations use Python callbacks through the GIL.

5.2.4 Licensing

Because POUNCE is a source-level translation of Ipopt (EPL-2.0), it is distributed under the Eclipse Public License 2.0 in its own repository (https://github.com/jkitchin/pounce). POUNCE replaced the earlier in-repo `ripopt` crate, which has been extracted into this standalone companion project. EPL-2.0 is not viral: code that /depends on/ POUNCE does not need to adopt EPL-2.0. discopt depends on POUNCE only through its Python bindings; we have additionally implemented a pure Python version in jax of the Ipopt algorithm in discopt, and have adopted the EPL-2.0 license for the entire discopt repository to avoid confusion and ensure that all components are covered by the same license.

5.3 Ipopt via cyipopt

The third backend uses the industry-standard Ipopt solver[15] via cyipopt Python bindings. This serves as the correctness reference: any discrepancy between the pure-JAX IPM or POUNCE and Ipopt on a well-conditioned problem indicates a bug in the former. Ipopt's advantage is its mature sparse linear algebra (MUMPS, MA27/MA57) which scales better on large, sparse problems ($n > 1000$). Its disadvantage is the Fortran/C++ dependency chain and inability to batch or differentiate through the solve.

6 Pure-Rust LP/MILP Simplex Engine

The linear subproblems that dominate the global solver — the McCormick/AMP LP relaxation solved at every spatial-B&B node, the MILP master of the AMP loop (Section 8), and standalone LP/MILP models — are handled by a self-hosted, pure-Rust revised simplex rather than an external library. A primal simplex handles cold and root solves; a bounded-variable *dual* simplex re-optimizes each B&B child from its parent's optimal basis, the warm start that makes node throughput competitive. The basis factorization is a pure-Rust sparse LU (the companion `feral` crate) with threshold partial pivoting and Forrest–Tomlin product-form updates between periodic refactorizations, so the engine carries no C or Fortran dependency and preserves the clean-wheel build.

Numerical robustness is handled *in-engine* rather than by swapping to a more robust external solver when a solve looks difficult. A numeric-focus mode adds iterative refinement of the basic solution and exposes element-growth and Hager–Higham condition estimates that gate a refined recompute of a drifted vertex before it is certified. Degeneracy — pervasive on the wide, lifted relaxations — is controlled by a Harris two-pass ratio test with an EXPAND expanding-tolerance minimum-step rule,[82] which reduces degenerate pivots by roughly 15× on the ill-conditioned lifted corpus, with a smallest-index (Bland) fallback that guarantees termination on a genuine cycle. Every `Optimal` is gated by an exact primal-feasibility audit and a Neumaier–Shcherbina safe dual bound, so a numerically drifted solve is downgraded to `Numerical` rather than trusted as a lower bound. This single hardened engine replaces an earlier fallback chain (external LP library → interior-point method → pure-JAX LP-IPM): the standalone and per-node LP paths now run on the Rust simplex alone, and POUNCE is retained for linear work only where an interior analytic-center point is required, such as the KKT sensitivity of the differentiable-LP layer (Section 10).

7 Convex Relaxation Framework

The quality of convex relaxations at B&B nodes directly determines solver efficiency. discopt provides a compositional relaxation compiler that walks the expression DAG and produces JIT-compatible relaxation functions.

7.1 McCormick Relaxation Compiler

The relaxation compiler implements 21 univariate and bivariate McCormick relaxation functions[18,19] covering all operations supported by the modeling API: bilinear products, addition, subtraction, negation, division, power, `exp`, square, `abs`, `sqrt`, `log`, `log2`, `log10`, `sin`, `cos`, `tan`, `sigmoid`, `softplus`, `sign`, `min`, and `max`.

Each function maps interval bounds $[x_l, x_u]$ and convex/concave enclosures (cv, cc) of the argument to tighter enclosures of the result. The compiler produces pure `jax.numpy` functions compatible with `jax.jit` and `jax.vmap`, enabling batch relaxation evaluation over multiple B&B nodes.

A uniform factorable relaxation engine. The per-operator relaxation table above is now driven by a single *uniform* factorable relaxation pass rather than a scattered per-operator

dispatch. The earlier design attached bespoke code to each operator and to a "federation" of specialised claimers; that machinery was consolidated into one compositional walk that lifts every intermediate factor of the expression DAG, attaches its rigorous (cv, cc) envelope from the shared rule table, and assembles a single McCormick LP over the lifted variables. The consolidation deleted a large body of production-dead specialisation code while leaving the bound bit-for-bit identical on the certifying panel (a bound-neutral refactor in the sense of Section 15), and it is the substrate on which the composite atoms and cut separators below compose. The resulting LP relaxation is solved on the in-house Rust revised simplex of Section **??**, not an external LP library, so an entire node — lift, relax, and solve — stays inside the hardened engine. A companion *adaptive* node-NLP policy (graduated default-on) tunes how aggressively each node's relaxation is re-solved.

Composite convex atoms. Beyond the scalar envelopes, the compiler recognises a set of multivariate *composite atoms* with known exact convex representations — log-sum-exp and softplus, the univariate entropy $x \log x$ and its relative-entropy generalisation, the Euclidean norm, and the convex $x\,e^x$ envelope on its convex domain. Each atom emits an exact convex outer approximation rather than the looser product-by-product McCormick fold, tightening the relaxation of the log-sum-exp, entropy, and norm structures that arise in geometric-programming and information-theoretic models.

7.2 α BB Underestimators

For general twice-differentiable functions where McCormick relaxations are loose, the α BB method[20,21] adds a separable quadratic perturbation:

$$L_\alpha(x) = f(x) - \sum_{i=1}^{n} \alpha_i (x_i - x_i^l)(x_i^u - x_i) \tag{2}$$

The parameters α_i are chosen such that L_α is convex: $\alpha_i \geq \max(0, -\lambda_{\min}(\nabla^2 f)/2)$ where $\lambda_{\min}$ is the minimum eigenvalue of the Hessian. discopt provides two estimation methods:

- **Eigenvalue method**: exact computation via `jax.hessian` and `jnp.linalg.eigvalsh` at sampled points (accurate but requires $O(n^3)$ per sample)
- **Gershgorin method**: conservative estimate from Hessian diagonal dominance (faster, slightly looser bounds, zero soundness violations in testing)

The JAX-native α BB estimation uses `jax.vmap` over sample points, providing 10–100× speedup over the finite-difference fallback used for `.nl` file models.

7.3 Piecewise McCormick Relaxations

Piecewise McCormick[22] partitions the variable domain into k subintervals and constructs tighter McCormick envelopes on each partition. discopt supports both uniform partitioning and adaptive partitioning that concentrates breakpoints where $f''(x)$ is largest.

On representative test functions, piecewise McCormick with $k = 8$ partitions achieves significant gap reduction versus standard McCormick:

Function	Standard gap	Piecewise gap	Reduction
$\exp(x)$	0.999	0.066	93.4%
$\log(x)$	0.254	0.021	91.9%
x^2	2.664	0.167	93.7%
bilinear xy	varies	varies	>30%

Table 2: Piecewise McCormick gap reduction with $k = 8$ partitions.

7.4 Cutting Planes: RLT, SOC, and Moment/PSD Separators

Envelope-based relaxations bound each nonlinear term in isolation; cutting planes tighten the relaxation by exploiting the **joint** structure of several terms. discopt separates four cut families, each as a JIT-compatible separator that returns violated inequalities at the current relaxation point:

- **Reformulation-linearization (RLT).**[46] Level-1 RLT multiplies pairs of constraint and bound factors and linearises the resulting products against the lifted bilinear variables, tightening the root bound on quadratically-constrained problems; the same products are also separated per node as targeted RLT cuts. The recursive McCormick fold for n-ary products is augmented by exact multilinear convex-envelope cuts[83] up to a capped arity.
- **Second-order-cone (SOC)** cuts. For a rotated or standard cone $\|y\|_2 \le t$, gradient (outer-approximation) cuts at the relaxation point provide a polyhedral outer approximation that sharpens the bound without a conic solver. The SOC and moment/PSD separators together target the quadratically-constrained relaxations that arise in alternating-current optimal power flow,[84] which serves as a capstone stress test for the cut framework.
- **Moment / PSD eigenvalue** cuts.[47] For box-QP and QCQP substructure, the lifted moment matrix $M(x, X) = \left(\begin{smallmatrix} 1 & x^\top \\ x & X \end{smallmatrix}\right)$ must be positive semidefinite; the separator computes its most negative eigenvalue and adds the corresponding eigenvector cut $v^\top M v \ge 0$, the first-order semidefinite relaxation.
- **Cover and clique** cuts for 0–1 knapsack substructure.

Rather than expose this machinery as a tuning burden, discopt defaults to a structure-gated policy: `Model.solve(cuts"auto")`= inspects the detected problem class and separates RLT on quadratically-constrained problems that carry linear constraints, PSD cuts on constraint-free box-QPs, and neither when no structure is present, so that the two strong but redundant families are never run together. An explicit `rlt_cuts`, `psd_cuts`, or =rlt=True/False= flag overrides the policy for controlled experiments, and `cuts`"manual"= disables it entirely.

Two further root-bound strengtheners target constrained binary and quadratically-constrained problems specifically. For a constrained binary quadratic program, an exhaustive level-1 RLT root bound is computed and, on the coupling rows, taken to its *Lagrangian dual* so that the

RLT multipliers are optimised rather than fixed; the resulting bound is passed through a Neumaier–Shcherbina safe-rounding step before it is trusted as a dual bound, and a bound-neutral set-partitioning exclusion presolve removes RLT products that a set-partitioning constraint already fixes. Separately, root *optimality-based bound tightening* (OBBT) iterates min/max probes of each variable over the McCormick LP relaxation to a fixpoint, tightening the box from which every downstream envelope is built; this is the QCQP-root complement to the FBBT of Section 4 and is exposed as part of the branch-and-reduce root pass. Both are held behind graduation flags until a differential panel certifies them cert-clean and net-positive.

An exact-linearisation route handles binary-multilinear structure without any envelope at all: a product of $\{0, 1\}$ variables (as in autocorrelation and Boolean-polynomial models) is replaced by its exact McCormick-tight linearisation, converting the node relaxation into an equivalent MILP that the Rust simplex/branch-and-bound solves to global optimality directly, with incumbent seeding to warm the search.

7.5 Learned Relaxations via ICNN

As an experimental feature, discopt supports learned convex relaxations using input convex neural networks (ICNNs).[23] An ICNN architecture with softplus non-negative weight constraints is trained to approximate tighter-than-McCormick underestimators for specific operations, with a soundness loss term that penalizes violations of the convexity and underestimation properties.

A `LearnedRelaxationRegistry` provides per-operation wrappers with automatic fallback to standard McCormick for unsupported operations. In testing, ICNN relaxations achieve approximately 10% gap reduction beyond standard McCormick, with the potential for 30%+ with more extensive training.

7.6 Structural Convexity Certification

Many problems written in unconstrained nonlinear form are in fact convex — a sum of squares, a log-sum-exp, an exponential cone, or an affine function with a convex regulariser — but the modelling layer cannot exploit that structure unless it is **proved**. discopt ships a sound convexity detector under `discopt._jax.convexity`, inspired by the disciplined convex programming (DCP) framework of Grant, Boyd and Ye[24] and the SUSPECT special-structure detector of Ceccon, Siirola and Misener.[25]

The detector classifies each (sub)expression by curvature (CONVEX, CONCAVE, AFFINE, or UNKNOWN) using two complementary, **sound** methods. The syntactic walker (`rules.py`) implements DCP composition rules: an affine combination of convex expressions with non-negative weights is convex; a non-decreasing convex function of a convex expression is convex; and so on. When the syntactic walker abstains, the box-local certificate (`certificate.py`) closes the gap by computing a sound interval enclosure of the Hessian over the variable box via interval automatic differentiation (`interval_ad.py`) and a sound lower bound on the minimum eigenvalue across that enclosure (`eigenvalue.py`, with both a Gershgorin disc bound and a tighter Schur-complement bound for arrow-structured Hessians); a non-negative lower eigenvalue bound proves convexity by the second-order sufficient condition.[21]

Symmetrically, a non-positive upper bound proves concavity. Any other outcome returns a conservative abstention — the certificate never claims nonconvexity, and never loosens a verdict from the syntactic walker.

Three downstream consumers benefit from a CONVEX verdict:

- **Convex fast path.** A model whose objective and all constraints certify CONVEX (or AFFINE) is routed to a single NLP solve with =SolveResult.convex_fast_path = True=, skipping spatial branch and bound entirely. For pure convex QPs this dispatches to POUNCE with a quadratic cost (the QP path is POUNCE-only), with the same status semantics as the general MINLP solve. Soundness of the certificate guarantees that the single NLP optimum is the global optimum.
- **Tighter αBB.** When the eigenvalue method (Section 7.2) is supported by a CONVEX certificate, the perturbation α can be set to zero on the certified terms, reducing relaxation gap.
- **Multi-objective scalarization** (Section #sec-mo) inherits the same convex fast path on each subproblem when the scalarised model certifies convex, e.g. weighted-sum on a sum of certified-convex objectives with non-negative weights.

The detector is conservative by design: it underclaims rather than overclaims. Every CONVEX or CONCAVE verdict is a mathematical proof, never a heuristic; sampling-based methods that could produce false positives are not used.

7.7 Geometric Programs

A geometric program (GP)[51,52] minimises a posynomial subject to posynomial-$\leq$-monomial and monomial-=-monomial constraints over strictly positive variables. A GP is generally **nonconvex** in the original variables, so a curvature certificate in x-space (Section 7.6) abstains; yet under the change of variables $y = \log x$ every monomial becomes the exponential of an affine form and every posynomial a convex sum of exponentials, rendering the whole program convex. discopt treats this as a distinct, sound verdict — **log-convexity** — computed by `discopt.gp.is_log_convex` without disturbing the x-space convexity detector.

When `classify_gp` recognises the standard GP form, `solve_model` routes the problem automatically through `as_geometric_program`, which builds the log-space convex NLP, solves it once with the convex fast path (a single global solve, no spatial branch and bound), and maps the optimiser back to x-space; soundness of the log-convex reformulation guarantees the recovered point is the global optimum. The routing is automatic for recognised GPs but can be forced with `solver`"gp"= or bypassed with `solver`"bb"=, and it is suppressed when streaming callbacks require the branch-and-bound path. Recognising log-convex structure converts an otherwise nonconvex global-optimisation problem into a single convex solve, the same payoff the convexity certifier delivers for x-space-convex models.

Generalised (G-)convexity. The log-space transform is one instance of a broader *generalised-convexity* recognition layer. A per-expression log-curvature lattice is propagated over the DAG, and a G-convexity detector applies an augmented-Hessian test to certify convexity under a monotone reparameterisation even when neither the x-space certifier nor the

strict GP recogniser fires. Where a sub-expression is G-convex (or, dually, log-concave) but the model as a whole is not, the layer emits rigorous *transformation-supporting* cuts — a signomial secant overestimator, a log-concave overestimator gated by a soundness proposition, and product/ratio envelopes in the transformed space — that tighten the node relaxation without disturbing the x-space bound. Each cut family is gated behind the differential-bound + feasible-point regime of Section 15 and injected only after passing a cert-clean panel.

GP-structured MINLP. When a geometric-program core carries genuine integer or binary variables, the pure log-space convex solve no longer applies, because the change of variables $y = \log x$ is defined only on the continuous positive block. discopt handles this *GP-structured MINLP* class by branching on the integers while solving each node's continuous relaxation in y-space: the integer variables drive a spatial/combinatorial branch-and-bound tree, and every node relaxation is the log-space convex program over the continuous GP block, certified by the same log-convex soundness argument. This routes an integer-augmented posynomial model to a sound global certificate rather than to a local NLP, and is gated behind a graduation-panel flag pending its net-positive graduation.

8 Adaptive Multivariate Partitioning (AMP)

In addition to the spatial branch and bound described in earlier sections, discopt provides AMP — a complementary global MINLP solver based on the Adaptive Multivariate Partitioning algorithm of Nagarajan and co-workers,[28,29] also known as the algorithm behind the Alpine solver. AMP is selected with `Model.solve(solver"amp")`= and is preferred when the bilinear / multilinear structure of the problem benefits from refined piecewise relaxations rather than from binary branching.

Where sBB enumerates a binary tree of variable-bound subregions, AMP iterates a single tightening loop on the **partitioning** of the continuous-variable domain. At iteration k the algorithm performs three steps:

1. Build a piecewise-McCormick MILP relaxation of the original MINLP using the current partition $\mathcal{P}^{(k)}$ of each branched variable, solve it to optimality, and obtain a lower bound LB_k together with a candidate point $\hat{x}^{(k)}$.

2. Fix each branched variable's interval assignment from $\hat{x}^{(k)}$ and solve the remaining continuous NLP to local optimality, obtaining an upper bound UB_k if feasible.

3. If $(UB_k - LB_k)/|UB_k| \le \varepsilon$, certify global optimality and stop. Otherwise refine $\mathcal{P}^{(k)}$ by splitting the partition cell containing $\hat{x}^{(k)}$ along its longest edge, increment k, and repeat.

Soundness is maintained at every iteration by the McCormick guarantee on the MILP relaxation:

$$LB_k \;\le\; \min_{x \in \mathcal{X}^{\mathrm{MINLP}}} f(x) \;\le\; UB_k \quad \text{for all } k, \tag{3}$$

so termination at the first k with closed gap is a certified ε-optimal solution. The MILP-relaxation construction is shared with the convex-hull / Big-M formulation work used elsewhere in discopt and is implemented in `discopt._jax.milp_relaxation`. The MILP itself is solved by the self-hosted pure-Rust simplex engine (Section **??**), and the NLP subproblem by any of the three NLP backends (default `pounce`; cyipopt is selected automatically when available).

Compared with the default sBB, AMP trades branching combinatorics for relaxation tightness: the MILP at each iteration is larger (one binary per partition cell per branched variable) but the tree is shallow because partitions, not subregions, are refined. On problems where the optimum lives on a few active partition cells — pooling problems, water-network synthesis, generalised disjunctive programs with bilinear terms — AMP closes the gap in tens of iterations where sBB would explore thousands of nodes.

The pooling problem[85,86] — blending streams of differing quality through intermediate pools to meet output specifications — is the canonical bilinear test of relaxation strength. The `discopt.pooling` module builds the **pq-formulation** via `build_pq_formulation`, which introduces proportion variables $q_{i\ell}$ (the fraction of pool ℓ's inflow drawn from input i) together with reformulation-linearization equalities $\sum_i w_{i\ell j} = y_{\ell j}$ for the bilinear products $w_{i\ell j} = q_{i\ell}\, y_{\ell j}$. These redundant-but-tightening constraints make the pq relaxation provably at least as tight as the textbook p- and q-formulations; on the classic Haverly instances (`haverly_hpp1`) the root relaxation is already exact, so the global optimum is certified at the root node.

v0.4 hardening. The first AMP release covered the core piecewise-McCormick / disaggregated-convex-hull path on bilinear and trilinear terms. The v0.4 line extends term coverage and relaxation tightness in four directions:

1. **Fractional-power lifting.** Fractional powers x^p with $p \in \mathbb{Q} \setminus \mathbb{Z}$ are lifted to MILP auxiliary variables with piecewise-secant under- and over-estimators, removing the previous restriction to integer monomial degrees.

2. **Universal piecewise-secant coverage.** Concave univariate terms (log, $\sqrt{\cdot}$, fractional powers, concave exponential branches) now receive piecewise-secant outer approximations consistent with the rest of the relaxation, rather than fall back to a single tangent.

3. **β-driven piecewise McCormick.** For products of the form $x{\cdot}y$ where one factor itself contains a fractional-power expression, AMP refines the partition along the variable that drives the largest envelope gap rather than the longest edge, giving faster gap closure on signomial pooling instances.

4. **Optimisation-based bound tightening on the relaxation.** An opt-in OBBT pass on the active MILP relaxation tightens variable bounds before each MILP solve. When an incumbent is available, the cutoff is added as a constraint during OBBT, and live partition state `disc_state` is passed through so the tightening reflects the current $\mathcal{P}^{(k)}$ rather than the root box.

The convex-hull formulation can be selected via =convhull_formulation $\in \{$`disaggregated`, `sos2`, `facet`$\}$= with the optional logarithmic Gray-code embedding =convhull_ebd=True=

(giving $\log_2 |\mathcal{P}|$ binaries per branched variable instead of $|\mathcal{P}|$). The algorithm also exposes `rel_gap`, `max_iter`, `time_limit`, `n_init_partitions`, `partition_method` (`adaptive`, default; `longest-edge`; `equal-bisection`), `presolve_bt`, `obbt_at_root`, `obbt_with_cutoff`, `alphabb_cutoff_obbt`, and an `iteration_callback` hook receiving `(iteration, lower_bound, upper_bound)` for live convergence inspection on `Model.solve(solver"amp", ...)`=.

9 Branch-and-Bound Search Enhancements

The lower bound at each node is only half of branch and bound; a strong incumbent prunes the tree, a good node ordering reaches it sooner, and learning from infeasible subtrees avoids repeating mistakes. discopt implements a set of search enhancements that, by construction, never weaken the global bound or the optimality certificate.

Primal heuristics. Beyond the feasibility pump,[63] `discopt._jax.primal_heuristics` provides three improvement heuristics that take the relaxation solution (and, where applicable, the incumbent) and return candidate integer-feasible points. **Diving** progressively fixes one fractional integer at a time and re-solves the node NLP, in either a fractional (nearest-integer) or objective-guided rounding mode. **RINS**[87] fixes every integer on which the incumbent and the relaxation agree and searches the small sub-MINLP over the disagreeing integers. **Local branching**[88] adds a k-flip Hamming-distance constraint around the incumbent and solves the resulting neighbourhood exactly. All three only propose incumbents; every bound mutation they make is temporary and restored, so they cannot affect the dual bound. They are opt-in and invoked at controlled points in the search rather than run unconditionally.

Node selection. In addition to best-first and depth-first, discopt offers a best-estimate strategy that orders open nodes by a pseudocost-style estimate of the integer optimum reachable below them, and an objective-gating priority rule that biases branching toward variables most likely to raise the dual bound. Because selection only reorders an exhaustive enumeration, any strategy preserves correctness; the strategies differ only in how quickly the gap closes.

Conflict analysis. When a node is proved infeasible, `discopt.conflict` analyses the responsible bound changes to extract a minimal infeasible assignment of binary variables and emits a no-good cut that excludes it tree-wide.[54] Because the excluded assignment is genuinely infeasible, the cut removes no feasible point and the global bound is unaffected.

Bound tightening as a public API. The feasibility-based bound tightening used internally in presolve is also exposed directly as `discopt.tightening.fbbt_box(model)`, which returns tightened variable bounds (or a proof of infeasibility when constraint propagation yields an empty interval) via row-activity interval arithmetic with no LP solve, supporting both standalone model analysis and reuse inside custom search loops.

Infeasibility analysis. For models that are proved infeasible, `discopt.infeasibility.compute_iis` returns an irreducible infeasible subsystem (IIS): a subset of constraints and variable bounds that is itself infeasible but becomes feasible if any single member is removed. The implementation uses deletion filtering,[53] tentatively dropping one member at a time and retaining it only when the remainder is **provably** infeasible. This guard makes the procedure sound under an incomplete solver: the returned set is always genuinely infeasible, and it is certified

irreducible exactly when every drop-probe was conclusive. Together with the convergence and limit diagnostics, the IIS turns a bare `INFEASIBLE` status into an actionable, human-readable explanation of which constraints conflict.

10 Differentiable Optimization

A distinguishing feature of discopt is the ability to differentiate through the optimization solve with respect to problem parameters. This enables decision-focused learning[11] where the model parameters (e.g., cost coefficients) are learned end-to-end from decision outcomes rather than prediction accuracy.

10.1 Level 1: Envelope Theorem

For a parametric NLP $\min_x f(x;p)$ subject to $g(x;p) \leq 0$, the envelope theorem gives:

$$\frac{df^*}{dp} = \left.\frac{\partial \mathcal{L}}{\partial p}\right|_{x^*,\lambda^*} = \left.\frac{\partial f}{\partial p}\right|_{x^*} + \lambda^{*T} \left.\frac{\partial g}{\partial p}\right|_{x^*} \tag{4}$$

This requires no additional linear system solve—the optimal dual variables λ^* from the NLP solver directly provide the sensitivity. discopt implements this as a `DiffSolveResult.gradient(para` method that returns df^*/dp for any model parameter.

10.2 Level 3: Implicit Differentiation via KKT

For the full primal sensitivity matrix dx^*/dp, discopt differentiates the KKT conditions:

$$\begin{bmatrix} \nabla^2_{xx}\mathcal{L} & J_a^T \\ J_a & 0 \end{bmatrix} \begin{bmatrix} dx^*/dp \\ d\lambda^*/dp \end{bmatrix} = - \begin{bmatrix} \nabla^2_{xp}\mathcal{L} \\ \partial g_a/\partial p \end{bmatrix} \tag{5}$$

where J_a is the Jacobian of active constraints (including active variable bounds) and $\nabla^2_{xp}\mathcal{L}$ is the mixed Hessian of the Lagrangian. This is computed via `jax.hessian` and `jax.jacobian` applied to parametric DAG-compiled functions.

The `DiffSolveResultL3` class provides:

- `sensitivity_matrix()`: full dx^*/dp matrix
- `implicit_gradient(param)`: df^*/dp via the chain rule $\nabla_x f \cdot dx^*/dp + \partial f/\partial p$
- `approximate_resolve(new_params)`: first-order approximation $x^* + (dx^*/dp)\Delta p$
- `dual_sensitivity(param)`: $d\lambda^*/dp$ for constraint multiplier sensitivity
- `reduced_hessian()`: Hessian projected onto the null space of active constraints (for uncertainty quantification)
- `sensitivity(dp)`: fast re-query with cached KKT factorization (analogous to sIPOPT[70])

If the KKT system is ill-conditioned, discopt falls back to finite-perturbation smoothing, and then to L1 envelope theorem gradients.

10.3 Differentiable Mixed-Integer Programs via Fix-and-Differentiate

The envelope and implicit-KKT sensitivities above assume a continuous solution manifold, which the integer variables of a MILP, MIQP, or MINLP break. discopt extends differentiability to the mixed-integer case through a **fix-and-differentiate** restriction: the problem is first solved to global optimality, the integer variables are then pinned at their optimal values, and the continuous restriction — now an ordinary parametric NLP — is differentiated by the Level 1 or Level 3 machinery above. This is exact wherever it matters: the optimal integer assignment is piecewise-constant in the parameters, so on the (open, full-measure) set where that assignment is locally stable the objective is smooth through the continuous restriction and df^*/dp is the true gradient; at the measure-zero breakpoints where the assignment switches, the procedure returns the one-sided sensitivity of the incumbent assignment, which is the appropriate convention for the gradient-descent updates of decision-focused learning.

A single entry point `differentiable_solve(model, method"auto")`= detects the problem class (LP, QP, NLP, MILP, MIQP, MINLP) and dispatches automatically, returning a `UnifiedDiffResult` that carries the optimal objective and solution, the detected `problem_class`, a `gradient(param)` method, and the continuous-relaxation objective for mixed-integer problems. The continuous paths reduce to the envelope/implicit-KKT results of the preceding subsections, so a user differentiating a model need not know in advance whether it contains integers.

10.4 Application: Decision-Focused Learning

The JAX-native differentiable solve is exposed via `jax.custom_jvp`, enabling it to be embedded in a larger JAX computation graph. A decision-focused learning pipeline:

1. A neural network $\hat{c} = \phi_\theta(x_{\text{features}})$ predicts cost parameters
2. The predicted costs are fed to `discopt.solve()` to obtain decisions $x^*(\hat{c})$
3. The true task loss $\ell(x^*(\hat{c}), c_{\text{true}})$ is computed
4. Gradients $\partial\ell/\partial\theta$ flow through the solver via L3 implicit differentiation
5. The network parameters θ are updated to minimize decision regret

On a resource allocation task with 3 activities, 5 features, and risk-averse objective, decision-focused learning with discopt achieves **36% regret reduction** compared to two-stage predict-then-optimize, despite having higher prediction MSE. This confirms the key insight: optimizing for decision quality beats optimizing for prediction accuracy.

On a mean-variance portfolio optimization task with 3 assets and nonlinear interactions, DFL similarly achieves lower regret and higher utility than the two-stage approach.

11 Dynamic Optimization

Dynamic optimization problems (optimal control, trajectory optimization, nonlinear model predictive control) are solved in discopt by transcribing ODE/DAE systems into algebraic

NLP constraints using orthogonal collocation on finite elements.[72,73] The `discopt.dae` module provides a `DAEBuilder` high-level interface: users declare state, algebraic, and control variables, supply an ODE/DAE right-hand side as a Python callable that builds discopt expressions, and call `discretize()` to generate all collocation and continuity constraints automatically.

Given a dynamic system $\dot{x} = f(x, z, u, t; \theta)$ with algebraic variables z satisfying $0 = h(x, z, u, t; \theta)$, each finite element $[t_k, t_{k+1}]$ of length h_k is transcribed by placing K collocation points $\tau_j \in (0, 1]$, introducing stage variables $x_{k,j}$, $z_{k,j}$, and enforcing

$$\sum_{l=0}^{K} \dot{\ell}_l(\tau_j)\, x_{k,l} = h_k\, f(x_{k,j}, z_{k,j}, u_k, t_{k,j}; \theta), \quad j = 1, \ldots, K, \tag{6}$$

$$0 = h(x_{k,j}, z_{k,j}, u_k, t_{k,j}; \theta), \tag{7}$$

$$x_{k+1,0} = \sum_{l=0}^{K} \ell_l(1)\, x_{k,l}, \tag{8}$$

where ℓ_l are Lagrange basis polynomials on the collocation nodes. Radau IIA (1–5 points) and Gauss-Legendre schemes are supported, along with automatic state augmentation for second-order ODEs and index-1 DAEs with algebraic constraints enforced at each collocation point. Piecewise-constant controls u_k and integral objectives $\int L(x, u, t)\, dt$ via Gauss quadrature cover standard optimal-control formulations. A finite-difference backend (`FDBuilder`) provides backward, forward, and central Euler stencils as a simpler alternative for users who prefer explicit time-stepping.

This transcription approach requires no changes to the existing DAG compiler, JAX backend, or solver infrastructure: the collocation constraints are standard algebraic expressions built from existing operators, so they inherit JIT compilation, `jax.vmap` batching, and the differentiable-solve machinery of the Differentiable Optimization section without special-casing. For a 30-element, 3-point Radau discretization of $\dot{x} = -x$ with $x(0) = 1$, the solution matches `exp(-t)` to within 10^{-4} absolute error on $[0, 10]$. The vectorized collocation evaluator (introduced in a recent NMPC-oriented perf update) computes stage-residuals for all elements and collocation points in a single JAX-batched kernel, eliminating Python-level loops over the horizon.

Because collocation reduces a dynamic problem to a structured sparse NLP, the same transcription machinery supports dynamic parameter estimation (Section 12) and dynamic experiment design (Section 13): the parameter vector θ enters f, h as an ordinary unknown, and the FIM through the dynamics is computed by the same JAX autodiff used for gradients.

12 Parameter Estimation

The `discopt.estimate` module estimates unknown model parameters from experimental data using weighted nonlinear least squares (equivalently, Gaussian-noise maximum likelihood).[89] Given responses y_i with measurement standard deviations σ_i, the estimation problem is

$$\hat{\theta} = \arg\min_{\theta \in \Theta} \sum_{i=1}^{N} \left(\frac{y_i^{\text{obs}} - y_i^{\text{model}}(\theta; d_i)}{\sigma_i} \right)^2, \tag{9}$$

where d_i are design conditions (sampling times, input concentrations, etc.) for observation i, Θ is a parameter box, and y_i^{model} is any discopt expression, including one produced by the DAE transcription of Section 11. Because the objective is an ordinary discopt expression, estimation reuses the full NLP stack: the pure-JAX IPM, POUNCE, or cyipopt.

Users declare an `Experiment` by subclassing a thin interface and providing `create_model(**kwargs)` returning an `ExperimentModel` that tags which variables are unknown parameters, which are design inputs, which expressions are responses, and the per-response measurement error. The same `Experiment` object is reused across estimation, FIM computation, and optimal experimental design, ensuring the three share a single source of truth.

```python
from discopt.estimate import Experiment, ExperimentModel, estimate_parameters

class ArrheniusExperiment(Experiment):
    def create_model(self, **kwargs):
        m = dm.Model("arrhenius")
        A  = m.continuous("A",  lb=1e3,  ub=1e8)
        Ea = m.continuous("Ea", lb=1e4,  ub=1e5)
        T  = m.continuous("T",  lb=300,  ub=500)
        k  = A * dm.exp(-Ea / (8.314 * T))
        return ExperimentModel(
            model=m,
            unknown_parameters={"A": A, "Ea": Ea},
            design_inputs={"T": T},
            responses={"k": k},
            measurement_error={"k": 0.05},
        )

result = estimate_parameters(ArrheniusExperiment(), data, initial_guess={"A": 1e6, "Ea": 5e4})
print(result.summary())  # parameters, standard errors, 95% CIs, FIM det and cond
```

The `EstimationResult` reports point estimates, the asymptotic covariance $\Sigma_\theta \approx (J^\top \Sigma^{-1} J)^{-1}$ where $J = \partial y^{\text{model}} / \partial \theta$, standard errors, confidence intervals via Student's t with $N - p$ degrees of freedom, and the correlation matrix. The Jacobian J is computed by `jax.jacobian` on the parametric DAG-compiled response functions, giving exact sensitivities (no finite-difference noise) at negligible overhead relative to the estimation solve. For dynamic models, the collocation transcription of Section 11 makes $y^{\text{model}}(\theta; d)$ a standard algebraic expression, so the same autodiff pipeline applies without special support for ODE sensitivity equations.

13 Model-Based Design of Experiments

Model-based design of experiments (MBDoE) selects experimental conditions d that maximize information content of the resulting data, quantified by the Fisher Information Matrix (FIM) $M(\theta, d) = J(\theta, d)^\top \Sigma^{-1} J(\theta, d)$, where $J = \partial y^{\text{model}} / \partial \theta$.[90,89] The `discopt.doe` module implements MBDoE on the same `Experiment` interface used for estimation (Section 12), so the model, responses, and measurement errors are shared across the two steps of a sequential design-then-estimate loop.

Compared with `Pyomo.DoE`,[91] discopt computes J by `jax.jacobian` applied to a parametric DAG-compiled response, giving exact sensitivities in a single forward-mode autodiff pass with no finite-difference step tuning, and supports GPU / TPU acceleration when available through the JAX backend.

Four optimality criteria are supported:

$$\text{D-optimal: } \max_d \log\det M(\theta, d), \qquad \text{A-optimal: } \min_d \operatorname{tr} M(\theta, d)^{-1}, \tag{10}$$

$$\text{E-optimal: } \max_d \lambda_{\min}(M(\theta, d)), \qquad \text{ME-optimal: } \min_d \kappa(M(\theta, d)), \tag{11}$$

evaluated with an optional prior FIM to support sequential designs. `optimal_experiment` performs a multi-start random + boundary search followed by scipy L-BFGS-B local refinement with bound constraints. `sequential_doe` alternates parameter estimation and single-experiment design over multiple rounds, updating the prior FIM with each round's contribution.

Batch / parallel MBDoE. When the laboratory can run multiple experiments in parallel (several reactors, wells, or robots), designing them one at a time wastes information. Because the experiments are independent, the Fisher information is additive,

$$M_{\text{total}}(d_1, \ldots, d_N) = \sum_{i=1}^{N} J(\theta, d_i)^\top \Sigma^{-1} J(\theta, d_i) + M_{\text{prior}}, \tag{12}$$

and the joint D-optimal batch design maximises $\log\det M_{\text{total}}$ over the stacked design vector $(d_1, \ldots, d_N)$. This joint-FIM formulation follows;[30] a modern framework for robust, uncertainty-aware parallel MBDoE is given in.[31] For an alternative route via parallel Bayesian optimization with Gaussian-process surrogates rather than FIM, see.[92]

The `batch_optimal_experiment` function exposes three strategies:

- **Greedy** (default). Pick one design via `optimal_experiment`, fold its FIM into the running prior, repeat. Cheap (N calls to the single-experiment search) and incremental, but myopic: each pick is locally optimal given the previously committed ones and is not guaranteed to recover the joint optimum on nonlinear experiments.

- **Joint**. Stack the N design vectors into $z \in \mathbb{R}^{Nd}$ and optimize the joint criterion directly via scipy L-BFGS-B with a multi-start seeding over random and quantile-structured stacks. Recovers the Galvanin[30] joint optimum up to local-solver convergence; on a two-parameter linear-Gaussian toy ($y = ax + b$, $x \in [0, 1]$) it reliably places $N = 2$ experiments at the boundary support $\{0, 1\}$ where greedy can miss depending on its first pick. Cost scales as $N \times d$ solver dimensions with repeated JAX autodiff passes, so it is slower than greedy for large N but equal or better in information content.

- **Penalized**. Greedy with a normalized minimum-distance filter between successive picks, useful when the criterion is locally flat and the experimenter wants spatial diversity in the batch.

All three strategies are also exposed as a `batch_strategy` option on `sequential_doe` via the `experiments_per_round` keyword, so a batched closed-loop campaign (Estimate → Design batch of N → Run N experiments → Estimate ...) is a single function call. The FIM prior accumulated across rounds ensures the batch criterion evaluated at each round reflects all previously collected data.

13.1 Identifiability and Estimability Diagnostics

Optimal-design criteria assume the model parameters are identifiable in the first place. When the data $\{y_i\}$ and the response Jacobian $J = \partial y^{\text{model}}/\partial\theta$ admit linear dependence between columns of J at the candidate $\hat{\theta}$, the FIM is rank-deficient or near-singular and individual standard errors blow up even though the model fit is excellent. The `discopt.doe` module addresses this with three complementary diagnostics, all built on the same DAG-compiled Jacobian used for FIM evaluation.

Structural identifiability. `compute_fim` returns the noise-weighted FIM and `check_identifiabilit` reports its numerical rank and null-space directions. A rank-deficient FIM means that any linear combination of parameters in the null space leaves y^{model} unchanged, i.e. those combinations are mathematically unidentifiable from the response model regardless of how much data is collected.

Estimability ranking. For practically identifiable subsets of an over-parameterised model we implement the orthogonalisation algorithm of Yao et al.[33] in `estimability_rank`, which iteratively selects the parameter whose scaled sensitivity column has the largest residual norm after Gram–Schmidt orthogonalisation against the previously selected columns. The complementary collinearity index γ_K of Brun, Reichert and K{\"u}nsch[34] (`collinearity_index`) measures the worst conditioning of any subset of size K: $\gamma_K = 1/\min_{\|\beta\|=1} \|Z_K\beta\|$ on the Brun-scaled sensitivity matrix $Z = \Sigma^{-1/2} J\,\mathrm{diag}(s_\theta)$, with the rule of thumb $\gamma_K \le 10$ for a practically identifiable subset. `d_optimal_subset` dispatches between exact enumeration (small p) and the greedy Yao ranking (larger p) following.[93]

Profile likelihood. For each parameter θ_j in turn, `profile_likelihood` steps θ_j outward from the global estimate $\hat{\theta}_j$, re-solves the estimation problem with θ_j fixed at each step, and tabulates the deviance

$$\Delta D(\theta_j) = D(\theta_j; \hat{\theta}^*_{-j}) - D(\hat{\theta}) \quad \text{vs.} \quad \chi^2_{1,1-\alpha}, \tag{13}$$

following Raue et al.[32] The intersection of the profile with the $\chi^2_{1,0.95}$ threshold gives a likelihood-ratio confidence interval that, unlike the asymptotic Wald interval, **retains its meaning when the FIM is ill-conditioned**: a profile that fails to cross the threshold in one or both directions diagnoses **practical non-identifiability** in that parameter; a flat profile signals **structural non-identifiability**. The deviance convention is fixed throughout (no factor of $\frac{1}{2}$), so the threshold is read directly against `result.objective`.

13.2 Model Discrimination and Selection

Once an experiment campaign has produced data that fits a chosen model, the natural next question is whether a **different** model would explain the data equally well, or whether

competing mechanistic hypotheses can be told apart by additional designs. discopt provides a model-discrimination MBDoE module under `discopt.doe.discrimination` and `discopt.doe.selection`, operating in **prediction space**: only $\hat{y}_i(d)$ and the prediction covariance $V_i = J_i\, M_i^{-1} J_i^\top$ from each candidate model cross model boundaries, so candidates may have entirely different parameter sets without any name alignment.

Five discrimination criteria are exposed via the `DiscriminationCriterion` enum:

- **HR** — Hunter–Reiner,[35] the squared difference of point predictions weighted by inverse measurement variance, $\Phi_{\mathrm{HR}}(d) = \sum_{i<j} (\hat{y}_i - \hat{y}_j)^\top \Sigma^{-1} (\hat{y}_i - \hat{y}_j)$. Cheap; pairwise; ignores prediction uncertainty.
- **BF** — Buzzi-Ferraris–Forzatti–Emig–Hofmann,[36] **the default**, generalising HR to multiresponse models by replacing Σ with $\Sigma + V_i + V_j$, so designs that put predictions far apart are penalised when the predictions are themselves uncertain.
- **JR** — Jensen–R{\'e}nyi divergence between Gaussian predictives $\mathcal{N}(\hat{y}_i, \Sigma + V_i)$ following Olofsson, Deisenroth and Misener.[37] Symmetric, naturally handles $M > 2$ candidates.
- **MI** — mutual information $I(M; y \mid d)$ between the model index and a future observation, estimated by nested Monte Carlo over the Gaussian predictives in the spirit of Lindley[38] and Foster et al.[39]
- **DT** — the DT-compound criterion of Atkinson, Bogacka and Bogacki,[40] $(1-\lambda)\Phi_{\mathrm{precision}}(M_p) + \lambda\Phi_{\mathrm{discrimination}}$, balancing parameter precision (any of the optimality criteria of Section 13 applied to the **preferred** model) against between-model separation. Useful when one model is provisionally preferred but the experimenter wants both to refine it and to keep its rivals in the picture.

The `discriminate_design` entry point picks one design under any of these criteria; `sequential_discrimination` alternates design with parameter re-estimation and updates a running posterior $p(M_k \mid \mathcal{D})$ over the candidate set via Akaike weights[94] or Bayes-factor approximation, terminating either at a posterior threshold (e.g. $\max_k p(M_k \mid \mathcal{D}) > 0.95$) or at a fixed budget of rounds. `model_selection` performs final-round selection by AIC / AIC_c / BIC / cross-validation given the accumulated data.

14 Multi-Objective Optimization

Many engineering problems trade off two or more conflicting objectives — cost vs. recovery, throughput vs. selectivity, capital vs. operating expense — and admit no single best solution but a **Pareto front** of mutually non-dominated alternatives. The `discopt.mo` module provides a scalarisation-based interface for tracing this front.[95,96] The design choice is deliberate: each scalarised subproblem is an ordinary discopt `Model`, so it inherits the full MINLP infrastructure — discrete variables, structural convexity certification (Section 7.6), spatial branch and bound, AMP global solving, the differentiable-solve API of Section 10,

and the McCormick / piecewise-McCormick / αBB relaxations. No separate evolutionary or surrogate-based MO machinery is needed.

Given K objectives $f_1, \ldots, f_K$ and a feasible set $\mathcal{X}$, a point $x^* \in \mathcal{X}$ is **Pareto-optimal** if no $x \in \mathcal{X}$ satisfies $f_k(x) \le f_k(x^*)$ for all k with strict inequality for at least one k. The five scalarisations exposed by `discopt.mo` are:

- **Weighted sum** (`weighted_sum`). Minimise $\sum_k w_k g_k(x)$ over a grid of non-negative weights summing to one, with $g_k = (f_k - z_k^*)/(z_k^N - z_k^*)$ if =normalize=True= (the default), where z^* is the ideal point and z^N the nadir point computed from the per-objective payoff table. Weighted sums are computationally cheap and recover the full convex Pareto front, but cannot reach points on a non-convex region of the front regardless of weight choice.

- **Augmented ε-constraint (AUGMECON2)** (`augmecon2`). Minimise $f_1(x) - \delta \sum_{k \ge 2} s_k$ subject to $f_k(x) + s_k = \varepsilon_k$, $s_k \ge 0$ for $k \ge 2$, sweeping ε_k over a grid in $[z_k^*, z_k^N]$ following Mavrotas.[41] The slack variables s_k together with the small augmentation δ guarantee that every solution returned is properly Pareto-optimal (no weakly-dominated points), and AUGMECON2's **bypass coefficient** skips ε grid points that the previous slacks have already shown to be redundant, sharply reducing the number of subproblems on dense grids.

- **Weighted Tchebycheff** (`weighted_tchebycheff`). Minimise $\max_k w_k(f_k(x) - z_k^*)$ via an epigraph reformulation $\min t$ s.t. $w_k(f_k - z_k^*) \le t$. Recovers all Pareto-optimal points (convex **and** non-convex regions), at the cost of a non-smooth max that becomes a single auxiliary variable plus K inequality constraints in the model.

- **Normal Boundary Intersection (NBI)** (`nbi`). Following Das and Dennis,[42] parameterise the front by points on the convex hull of the per-objective anchors and project orthogonally toward the lower-left, producing a uniform spread along the Pareto surface for problems where uniformly-spaced weights produce clustered solutions.

- **Normalised Normal Constraint (NNC)** (`nnc`). The complementary scheme of Messac, Ismail-Yahaya and Mattson,[43] which slides a hyperplane normal across the normalised utopia line, yielding well-spread fronts on smooth bi- and tri-objective problems.

The `ParetoFront` container collects the resulting non-dominated points along with their objective values and original-space variable assignments; `filter_nondominated` post-processes any candidate set into the Pareto subset.

For comparing two front approximations, `discopt.mo.indicators` provides four standard quality measures[44,45] — the **hypervolume** (exact 2-D and 3-D Lebesgue measure of the dominated region behind a reference point, Monte-Carlo for $K \ge 4$), the **inverted generational distance** (mean distance from a reference front to the approximation, capturing both convergence and spread in one number), the **spread** (coefficient of variation of consecutive point distances in objective space, lower is more uniform), and the additive **epsilon**

indicator $I_\varepsilon(A, B) = \min \varepsilon$ such that $A \preceq_\varepsilon B$ in the dominance order. All indicators internally convert to the all-minimise convention using the per-objective senses stored on the front, so results are sense-invariant.

Out of scope in the current release and left to future work: evolutionary MO algorithms (NSGA-II, MOEA/D), Bayesian MO acquisition functions over Gaussian-process surrogates, and interactive / preference-articulation methods that elicit the decision-maker's value function during the search rather than offering a static front.

15 Certificate Soundness

A global solver's product is not a number but a *certificate*: a claim that the returned point is globally optimal (or that the model is infeasible) backed by a valid dual bound. A change that makes the solver faster but admits a false `optimal`, false `infeasible`, or invalid bound on any instance is a regression, not an improvement. We therefore treat soundness as a first-class deliverable and report a systematic audit of the certificate, conducted as roughly three dozen tracked correctness issues (each with a deterministic reproducer, a fix, and a regression test) together with a twenty-module code review of the whole codebase. The governing invariant throughout is =incorrect_count = 0= on the certifying panel — a hard gate with zero slack that no fix is permitted to weaken.

The issues cluster in the layers where an unsound step silently corrupts the bound:

- **Branch-and-bound status logic.** A nonconvex model whose root cannot be spatially branched (a variable with a one-sided or fully free domain) and for which no relaxation ever established a finite dual bound was falsely certified `optimal` with zero gap on a local incumbent, because the tree collapsed the global lower bound onto the incumbent. The fix makes the tree carry the unresolved-bound condition explicitly, so such a root reports `feasible`*unknown* — *it /abstains* from the optimality claim — and never a false optimal. Related fixes require a rigorous proof (not a mere NLP-solve failure) before fathoming a node infeasible, and refuse to trust an under-converged NLP objective as a node lower bound.

- **Relaxation math.** Several McCormick envelopes were made sound on their domain edges: `relax_tan` now abstains to the no-information envelope across a pole rather than drawing a secant through it; `relax_div` falls back to a sound interval enclosure for a nonlinear denominator instead of an invalid convex underestimator; `relax_asin` / `relax_acos` use the correct curvature regime; and infinite-bound boxes degrade to the sound no-information bracket rather than producing `NaN`. The αBB node bound was routed through a rigorous per-box α and now abstains whenever a rigorous α cannot be certified, replacing an unsound sampled-α / centre-only positive-semidefinite check.

- **Presolve and FBBT.** Feasibility-based bound tightening on an array-valued variable block previously collapsed the block to a single element's bounds and stamped them on every element, which could both cut feasible points and report false infeasibility; it now seeds each block from the element-wise *union* of bounds, a valid outer box. A presolve bound-resync after a variable-removing pass no longer fuses the bounds

of unrelated variables. Interval multiplication that produced `NaN` on the $0 \cdot \infty$ corner (a lost-tightening bug, never an unsound verdict) was closed on both the Rust and Python paths.

- **Cuts and disjunctions.** A no-good / outer-approximation cut is emitted only on rigorously proven infeasibility; Gomory-style cuts carry a right-hand-side safety margin; and a Farkas ray is verified before an LP-infeasible fathom. In the generalized-disjunctive layer, a big-M reduction that treated a large-but-finite default bound as effectively infinite and substituted a far-too-small M — turning an inactive disjunct's constraint into a real one and producing a false `infeasible` on the *default* solve path — was corrected to use a finite bound as a valid M and to refuse loudly when no finite M exists. An even-power bound over a zero-straddling base now includes the interior minimum.

- **The `.nl` parser and interop.** The MINLPLib `.nl` import path was hardened to inline defined variables, handle additional opcodes, preserve integer and binary variable bounds, and refuse (rather than silently misparse) unsupported `floor`/`ceil`/`round` constructs; the Jacobian sparsity structure was made ASL-conformant.

- **The LLM front-end.** An expression-evaluation path in the (optional) natural-language model-builder that used Python `eval` was replaced with a whitelisted-abstract-syntax-tree evaluator, closing a remote-code-execution class while preserving the safety invariant that LLM outputs never affect solver math.

The audit also produced affirmative *soundness confirmations* where the review found no defect: the structural convexity certifier and convex fast path — the highest-stakes gate, since a wrong convex verdict would certify a nonconvex model's local optimum as global — were verified sound under a nine-hundred-plus-certificate randomized fuzz with zero false verdicts, as were the primal/dual simplex, the branch-and-bound tree manager, the PyO3 boundary, and the RLT/PSD/SOC/edge-concave cut families. The recurring discipline across every fix is the same: where a rigorous guarantee cannot be established, the solver *abstains* (returns a weaker but valid status or bound) rather than emitting an unsound certificate.

16 Computational Experiments

16.1 NLP Backend Comparison

We compare the three NLP backends on 5 test problems spanning unconstrained quadratics to constrained portfolio optimization. Table 3 reports objectives and wall-clock timing. All timings were collected on an Apple M4 Pro (CPU backend, JAX 0.8.2) and represent single-solve wall-clock time including model compilation.

Key observations:

- All three backends produce **identical optimal objectives** to solver tolerance (10^{-6}), confirming correctness across the Rust, JAX, and C++ implementations.

Problem	f^*	IPM (s)	POUNCE (s)	Ipopt (s)
Quadratic ($n = 2$)	0.000000	0.023	0.033	0.013
Constrained ($n = 5$)	1.250000	0.269	0.001	0.001
Rosenbrock ($n = 2$)	0.000000	0.590	0.082	0.114
Sum-of-squares ($n = 10$)	0.000000	1.189	0.049	0.070
Portfolio ($n = 8$)	0.002064	0.229	0.004	0.001
Total		2.30	0.17	0.20

Table 3: NLP backend comparison. All backends achieve identical objectives (to solver tolerance). Timings on Apple M4 Pro, CPU.

- POUNCE is competitive with Ipopt (total 0.17s vs 0.20s), and faster on Rosenbrock (0.082s vs 0.114s) and sum-of-squares (0.049s vs 0.070s).
- The pure-JAX IPM includes JIT compilation overhead on first evaluation; subsequent solves of similarly-structured problems benefit from cached JIT traces.
- On a quartic constrained NLP ($n = 10$), POUNCE (0.334s) and the JAX IPM (0.795s) agree to 6.5×10^{-15} in objective value, confirming numerical equivalence.

16.2 MINLP Correctness

We validate on 24 MINLP problems (12 NLP + 12 MINLP with binary/integer variables) from the discopt test suite, with known optimal values. This panel, together with the vendored MINLPLib instances below, forms the *certifying panel* on which the =incorrect_count = 0= gate of Section 15 is enforced — every soundness fix in the audit is required to hold this gate:

- **24/24 solved correctly** with zero incorrect results
- Tolerances: absolute 10^{-4}, relative 10^{-3} for objectives; integrality tolerance 10^{-5}
- Problems include Rosenbrock, exponential, logarithmic, trigonometric, bilinear, and cardinality-constrained formulations

B&B convergence with the POUNCE backend was verified on two MINLP instances: a simple 3-variable problem (optimal $f^* = 2.0$, 1 node, 0.039s) and a larger 6-variable formulation (optimal $f^* = 3.0$, 13 nodes, 2.38s).

16.2.1 MINLPLib Validation

We further validate on instances from MINLPLib,[81] imported via the `.nl` file parser. Table 4 reports results on a representative 10-instance smoke test suite. (This early smoke test predates the pure-Rust LP core of Section **??** and the relaxation-coverage work of Section 7, which certify many more of these instances; the head-to-head below is the current measurement.)

Head-to-head against BARON and SCIP. We compare discopt against two state-of-the-art global solvers — full-license BARON[1] run through GAMS and SCIP[2] — on a panel of roughly two hundred MINLPLib `.nl` instances spanning convex and nonconvex MIQP, nonconvex MINLP, and nonconvex NLP, under a matched 60~s per-instance budget and 10^{-4} relative gap tolerance, taking the MINLPLib primal bound as the oracle. The comparison separates cleanly into two axes — a *soundness* axis, on which discopt is at parity, and a *coverage* axis, on which it trails.

Soundness. discopt records **zero correctness violations** against either baseline: across the whole panel, no run ever certified a global optimum at the wrong value or returned an incumbent strictly better than the proven optimum. Every instance on which discopt claims a certified global matches the oracle to the 10^{-4} tolerance. This is the deliverable the certificate audit of Section 15 exists to protect, and it holds against decades-tuned commercial and academic baselines alike.

Coverage. On the fraction of the panel a solver *certifies* to global optimality within the budget, the two mature solvers lead. Judged by objective-versus-oracle, BARON and SCIP each certify on the order of 180 of the ≈ 200 instances, while discopt certifies roughly 120. We report BARON's rate by objective-versus-oracle rather than by GAMS return code deliberately: the GAMS status channel undercounts BARON's true solve rate by roughly a factor of two, so a status-code tally would spuriously flatter discopt. We therefore make **no** claim that discopt is more rigorous or higher-coverage than BARON — an earlier reading to that effect was a measurement artifact of the status-code undercount, not a real result. On the instances where discopt does not certify within the budget it almost always still reaches a *feasible* incumbent; the shortfall concentrates on harder medium-sized nonconvex instances (the `casctanks`, `heatexch_gen*`, `contvar`, and `tls2` families among them) where a serial per-node engine cannot close the optimality gap in 60~s. Crucially this is a wall-time / gap-closure limitation, not a correctness or relaxation-validity one, and it is the principal open performance work, tracked as issues #727, #723, and #282.

The honest headline is therefore: discopt's certificate is **sound everywhere it is exercised** and it is competitive on the classes it does solve, but it **trails BARON and SCIP on coverage of the hardest instances** under a fixed time budget — a per-node wall-clock gap on a serial engine, not a soundness or bound-validity deficit.

Measured discopt coverage. As a concrete, reproducible snapshot of the discopt side, a 116-instance global-optimisation panel drawn from the same corpus (2026-07-17 run, 60~s budget, Apple M4 Pro) resolves to 87 instances certified globally optimal, 18 feasible-but-uncertified incumbents, and 11 with no incumbent returned in the budget, with **zero** correctness violations on the subset carrying a published oracle value (32/32 certified optima matched the oracle to tolerance). The $\approx 75\%$ certified-optimality rate on this panel is higher than on the harder ≈ 200-instance head-to-head above, reflecting the larger share of medium and hard nonconvex instances in the latter.

Of the 9 solvable instances, 7 reach optimal status (matching known optima to 10^{-3}), while 2 reach feasible solutions at the time limit. The `gear` instance finds a near-zero objective (2.3×10^{-11}, matching the known optimum 0.0) but, **at the time of this smoke test**, could not prove optimality because the `.nl` import path did not yet build convex relaxations; instance `nvs01` reached a local optimum for the same reason. The JAX computation accounts for 92–99.9% of solver time, with Rust tree management contributing $< 1\%$ and

Instance	n	m	Status	f^*	Best Known	Time (s)	Nodes
chance	4	3	optimal	29.421	29.894	0.244	0
dispatch	4	2	optimal	3155.288	3155.288	0.258	0
ex1221	5	5	optimal	7.667	7.667	0.010	1
ex1226	5	5	optimal	-17.000	-17.000	0.176	3
gear	4	0	feasible	2.3e-11	0.000	60.000	25,153
nvs01	3	3	feasible	-4.904	12.470	60.731	7,787
nvs03	2	2	optimal	16.000	16.000	0.036	11
nvs04	2	0	optimal	0.720	0.720	0.078	9
nvs06	2	0	optimal	1.770	1.770	0.022	9

Table 4: MINLPLib smoke test results (10 instances, 60s time limit). Best known objectives from MINLPLib.

Python overhead $< 2\%$.

Subsequent to this table, the `.nl` import path was given the full relaxation and dual-bound machinery of Sections 7 and 7.4, so the solver now closes the gap and certifies global optimality on the majority of these instances rather than stopping at a feasible point. We have validated this expanded path head-to-head against BARON[1] on the subset of MINLPLib instances with a proven global optimum, matching the known optimum (to the 10^{-4} correctness tolerance) with zero incorrect results; the residual cases where discopt finds the optimum but does not yet prove it are concentrated in high-degree factorable products whose lifted relaxation is loose, consistent with the limitation noted in the Discussion.

16.3 Neural Network Embedding Validation

We validate the `discopt.nn` module on feedforward networks with ReLU and smooth activations. A representative 2-input, 4-hidden, 1-output network demonstrates the three formulation strategies:

- **ReLU big-M**: 4 binary variables (one per hidden ReLU neuron), 9 continuous variables, 16 linear constraints. Interval arithmetic propagates tight pre-activation bounds, simplifying neurons whose bounds are entirely positive or negative to linear constraints with no binary variable.
- **Full-space** (tanh): 9 continuous variables, 8 nonlinear activation constraints. McCormick relaxations applied to tanh activation functions provide valid lower bounds within spatial B&B.
- **Reduced-space**: zero intermediate variables; nested expressions build a single composite function, suitable for networks with ≤ 3 layers.

All three strategies produce identical optimal objectives on the test instance, confirming correctness of the algebraic encoding. The ReLU big-M formulation solves to global optimality at the root node (no branching required) when pre-activation bounds are tight enough to

fix all binary variables; the full-space formulation requires 1–3 B&B nodes with McCormick relaxations. The ONNX reader is validated against PyTorch-exported models: imported weights and biases match to machine precision, and `NetworkDefinition.forward()` reproduces the PyTorch output to 10^{-14} absolute error.

16.4 Relaxation Gap Reduction

Table 2 reports piecewise McCormick gap reduction. Additionally:

- α BB underestimators achieve zero soundness violations across 1000+ test points with both eigenvalue and Gershgorin methods
- ICNN learned relaxations provide approximately 10% additional gap reduction beyond standard McCormick with lightweight training
- The relaxation compiler supports mode switching between `standard`, `piecewise`, and `learned` at the solver level

16.5 Performance

Algebraic coefficient extraction. For LP and QP subproblems, walking the expression DAG directly to extract coefficient matrices avoids JAX tracing and autodiff:

- LP ($n = 100$): 3.5s $\rightarrow$ 0.20s warm (17$\times$ speedup)
- QP ($n = 100$): 65.7s $\rightarrow$ 0.29s warm (226$\times$ speedup)

Batch evaluation via `vmap`. For batch size 512 with $n_{\text{vars}} = 50$, `jax.vmap` achieves approximately 29.6$\times$ speedup over sequential evaluation.

Interop overhead. Rust $\leftrightarrow$ Python array transfer via PyO3/numpy: sub-microsecond for small batches, 0.03–0.14% overhead relative to solver time (well under the 5% performance gate).

Time breakdown. Across the MINLPLib smoke test, JAX numerical computation dominates at 92–99.9% of wall time. Rust tree management accounts for 0.05–7.3%, and Python orchestration overhead is 0.02–0.6%. This confirms that the multi-language architecture introduces negligible overhead.

JAX-free linear and quadratic paths. The 92–99.9% JAX share above is specific to nonlinear models. Because JAX import and first-trace compilation impose a fixed cold-start cost (~0.5–1 s) that dominates small solves, the LP, MILP, QP, and MIQP paths are now kept JAX-free: the relaxation compiler, NLP evaluator, and αBB modules are imported lazily and only when a genuinely nonlinear relaxation is built. MIQP node relaxations are solved as quadratic programs through POUNCE rather than the JAX QP IPM, so an integer-quadratic solve never touches JAX. The result is that a purely linear or quadratic model pays none of the cold-start tax that the differentiable nonlinear path requires.

Simplex pivoting. The Rust dual simplex used for LP and MILP node relaxations replaces Dantzig pricing with dual Devex reference-weight pricing,[97] a steepest-edge approximation that reduces the dual iteration count, and replaces the single-breakpoint dual ratio

test with a bound-flipping (long-step) ratio test that resolves more infeasibility per pivot by flipping finite-bounded nonbasic variables to their opposite bound as it walks the breakpoints. Both are standard ingredients of production simplex codes and are paired with the LP equilibration and shared-factorization reuse described earlier to keep the LU well-conditioned across a node's strong-branching probes.

Per-node subsolver: re-profile and a bound-neutral lever. A re-profile of the spatial-B&B nodes on nonconvex instances identified the *per-node subsolver* — not tree management or Python orchestration — as the dominant wall-clock cost, and localised it to the LP solves issued by the cut separators and by strong branching. On the `nvs17` instance these families issued on the order of 240 cold interior-point LP solves ($\approx$ 40% of wall time) that the in-house warm-started Rust simplex resolves in a few milliseconds each. Routing both LP families to the warm simplex is a *bound-neutral* change — the separator consumes only the LP's dual slope and recomputes the cut intercept to the exact validity boundary, and strong branching reads only the child objective bound, never the vertex — and it was verified neutral on the certification baseline: node counts were *exactly* unchanged across the panel, objectives bit-for-bit identical, and =incorrect_count = 0=. The measured effect is that `nvs17` moves from a 60~s time-limit miss (feasible) to a certified optimum in 38~s (per-node time roughly halved, $\approx 2\times$), and a companion instance `nvs13` from 3.9~s to 1.4~s ($\approx 2.7\times$); instances that issue no separation or strong-branch LPs are unaffected. The residual per-node cost was traced to the incumbent NLP subsolves: an apples-to-apples replay of the same subproblems from the same starting points found the in-house POUNCE backend roughly an order of magnitude slower than Ipopt (pooled median $\approx 7\times$; $\approx 11\times$ on the subset reaching the same optimum), a POUNCE performance problem with a minimal reproducer that has been filed for upstream work rather than papered over.

17 Discussion

discopt demonstrates that a modern, multi-language MINLP solver can provide capabilities absent from existing monolithic solvers: differentiability, batch evaluation, and memory safety. Several limitations and directions for future work deserve discussion.

Limitations. The pure-JAX IPM uses dense linear algebra, which limits scalability to problems with $n \lesssim 5000$ variables. For larger problems, the sparse IPM backend (using scipy sparse factorization) or Ipopt should be used. The `.nl` import path now constructs convex relaxations and surfaces a valid dual bound (the earlier smoke-test table predates this capability), so global optimality can be certified on many imported MINLPLib instances; the remaining gap is concentrated in high-degree factorable structure, where the lifted relaxation can be loose or expensive to build. The JAX Metal backend (Apple GPU) is currently broken in JAX 0.8.2, so GPU acceleration is CPU-only on macOS.

Comparison with existing solvers. discopt is not intended to replace BARON or SCIP on their core strength — solving large-scale MINLP to global optimality via decades-tuned sparse algorithms. The head-to-head of Section 16 is candid about the standing: on *coverage* of a hard $\approx$ 200-instance MINLPLib panel under a fixed 60~s budget, BARON and SCIP each certify on the order of 180 instances to discopt's $\approx$ 120, and discopt's shortfall is a per-node wall-clock / gap-closure gap on a serial engine rather than an unsound bound.

What discopt establishes instead is (i) *soundness parity* — zero correctness violations against both mature baselines across the whole panel — and (ii) a complementary capability set absent from monolithic solvers: differentiability, batch node evaluation, memory safety, and Python-native modeling. Closing the coverage gap is active performance work (issues #727, #723, #282); the multi-backend architecture also lets users start with the pure-JAX IPM for rapid prototyping and differentiable workflows, then switch to POUNCE or Ipopt for production performance.

Command-line interface and literature monitoring. discopt provides a command-line interface (`discopt`) installable via `pip`, with subcommands for searching arXiv and OpenAlex APIs and writing structured reports. These subcommands output JSON, enabling integration with scripting workflows and CI pipelines. A companion "discoptbot" agent uses the CLI to periodically scan for new publications relevant to the solver's algorithmic components (spatial B&B, McCormick relaxations, GNN branching, GPU-accelerated IPM), score them by relevance, and produce structured literature reports. This automated monitoring helps maintain awareness of algorithmic advances that could inform future solver development.

Future work. Key directions include: (1) GPU-accelerated batch IPM via JAX's XLA backend when GPU support matures; (2) extending POUNCE with sparse linear algebra to compete with Ipopt on large problems, and addressing the per-solve POUNCE-versus-Ipopt gap localised in the performance re-profile; (3) deeper integration of learned relaxations with online training during the B&B search; (4) meta-learning of solver parameters through the differentiable solver, and extending the bilevel module (Section 4.8) to integer and nonconvex lower levels beyond the current convex-KKT reduction; (5) extending the DAE collocation module to higher-index DAEs, adaptive mesh refinement, and multi-stage optimal control with path constraints; (6) extending the neural network embedding to convolutional layers, residual connections, gradient-boosted tree models, and LP-based bound tightening (OBBT) for tighter big-M constants; and (7) extending the uncertainty stack (Sections 4, 4.7) beyond the current affine decision rules and two-stage recourse — to bilinear coefficient uncertainty, polynomial or piecewise-linear recourse policies, multistage (nested-Benders / SDDP) recourse, and a full single-level distributionally-robust reformulation over moment-based or Wasserstein ambiguity sets.[77,79]

18 Conclusion

We have presented discopt, a hybrid Rust/JAX/Python MINLP solver that bridges the gap between classical mathematical programming and modern differentiable programming. The solver provides three interchangeable NLP backends—a pure-JAX IPM, POUNCE (a memory-safe Rust translation of Ipopt, developed as a separate companion project), and Ipopt via cyipopt—all producing identical results on benchmark problems. The differentiable solving capability enables decision-focused learning through nonconvex optimization, achieving 36% regret reduction on resource allocation tasks. The compositional relaxation compiler achieves 91–94% gap reduction via piecewise McCormick, and the algebraic coefficient extraction path provides 17–226× speedups for structured subproblems. The neural network embedding module enables optimization over trained ML surrogates by formulating feedfor-

ward networks as MINLP constraints with big-M, full-space, and reduced-space strategies. POUNCE eliminates the Fortran/C++ dependency chain while preserving Ipopt's numerical robustness.

The system is open-source: discopt is available under the EPL-2.0 License, and POUNCE is available as a separate companion project (https://github.com/jkitchin/pounce) under the EPL-2.0 License.

References

[1] Mohit Tawarmalani and Nikolaos V. Sahinidis. A polyhedral branch-and-cut approach to global optimization. *Mathematical Programming*, 103(2):225–249, 2005. doi: 10.1007/s10107-005-0581-8.

[2] Ksenia Bestuzheva, Mathieu Besançon, Wei-Kun Chen, Antonia Chmiela, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Oliver Gaul, Gerald Gamrath, Ambros Gleixner, Leona Gottwald, Christoph Graczyk, Katrin Halbig, Alexander Hoen, Christopher Hojny, Rolf van der Hulst, Thorsten Koch, Marco Lübbecke, Stephen J. Maher, Frederic Matter, Erik Mühmer, Benjamin Müller, Marc E. Pfetsch, Daniel Rehfeldt, Steffan Schlein, Franziska Schlösser, Felipe Serrano, Yuji Shinano, Boro Sofranac, Mark Turner, Stefan Vigerske, Fabian Wegscheider, Philipp Wellner, Dieter Weninger, and Jakob Witzig. Enabling research through the SCIP optimization suite 8.0. *ACM Transactions on Mathematical Software*, 49(2):1–21, 2023. doi: 10.1145/3585516.

[3] Pietro Belotti, Christian Kirches, Sven Leyffer, Jeff Linderoth, James Luedtke, and Ashutosh Mahajan. Mixed-integer nonlinear optimization. *Acta Numerica*, 22:1–131, 2013. doi: 10.1017/S0962492913000032.

[4] Ignacio E. Grossmann. *Advanced Optimization for Process Systems Engineering*. Cambridge University Press, 2021. ISBN 978-1-107-18920-5. doi: 10.1017/9781108917834.

[5] Ailsa H. Land and Alison G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960. doi: 10.2307/1910129.

[6] R. J. Dakin. A tree-search algorithm for mixed integer programming problems. *The Computer Journal*, 8(3):250–255, 1965. doi: 10.1093/comjnl/8.3.250.

[7] E. M. B. Smith and C. C. Pantelides. A symbolic reformulation/spatial branch-and-bound algorithm for the global optimisation of nonconvex MINLPs. *Computers & Chemical Engineering*, 23(4–5):457–478, 1999. doi: 10.1016/S0098-1354(98)00286-5.

[8] Pierre Bonami, Lorenz T. Biegler, Andrew R. Conn, Gérard Cornuéjols, Ignacio E. Grossmann, Carl D. Laird, Jon Lee, Andrea Lodi, François Margot, Nicolas Sawaya, and Andreas Wächter. An algorithmic framework for convex mixed integer nonlinear programs. *Discrete Optimization*, 5(2):186–204, 2008. doi: 10.1016/j.disopt.2006.10.011.

[9] Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine*

Learning (ICML), pages 136–145, 2017. URL https://proceedings.mlr.press/v70/amos17a.html.

[10] Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and J. Zico Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/9ce3c52fc54362e22053399d3181c638-Abstract.html.

[11] Adam N. Elmachtoub and Paul Grigas. Smart "Predict, then Optimize". *Management Science*, 68(1):9–26, 2022. doi: 10.1287/mnsc.2020.3922.

[12] Bryan Wilder, Bistra Dilkina, and Milind Tambe. Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1658–1665, 2019. doi: 10.1609/aaai.v33i01.33011658.

[13] Jayanta Mandi, Emir Demirović, Peter J. Stuckey, and Tias Guns. Smart predict-and-optimize for hard combinatorial optimization problems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 1603–1610, 2020. doi: 10.1609/aaai.v34i02.5521.

[14] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/jax-ml/jax.

[15] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006. doi: 10.1007/s10107-004-0559-y.

[16] Nicholas D. Matsakis and Felix S. Klock. The Rust language. *ACM SIGAda Ada Letters*, 34(3):103–104, 2014. doi: 10.1145/2692956.2663188.

[17] Sanjay Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2(4):575–601, 1992. doi: 10.1137/0802028.

[18] Garth P. McCormick. Computability of global solutions to factorable nonconvex programs: Part I — Convex underestimating problems. *Mathematical Programming*, 10 (1):147–175, 1976. doi: 10.1007/BF01580665.

[19] Angelos Tsoukalas and Alexander Mitsos. Multivariate McCormick relaxations. *Journal of Global Optimization*, 59(2–3):633–662, 2014. doi: 10.1007/s10898-014-0176-0.

[20] Ioannis P. Androulakis, Costas D. Maranas, and Christodoulos A. Floudas. αBB: A global optimization method for general constrained nonconvex problems. *Journal of Global Optimization*, 7(4):337–363, 1995. doi: 10.1007/BF01099647.

[21] Claire S. Adjiman, Ioannis P. Androulakis, and Christodoulos A. Floudas. A global optimization method, αBB, for general twice-differentiable constrained NLPs — II. Implementation and computational results. *Computers & Chemical Engineering*, 22(9): 1159–1179, 1998. doi: 10.1016/S0098-1354(98)00218-X.

[22] Pedro M. Castro. Tightening piecewise McCormick relaxations for bilinear problems. *Computers & Chemical Engineering*, 72:300–311, 2015. doi: 10.1016/j.compchemeng.2014.03.025.

[23] Brandon Amos, Lei Xu, and J. Zico Kolter. Input convex neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 146–155, 2017. URL https://proceedings.mlr.press/v70/amos17b.html.

[24] Michael Grant, Stephen Boyd, and Yinyu Ye. Disciplined convex programming. In Leo Liberti and Nelson Maculan, editors, *Global Optimization: From Theory to Implementation*, pages 155–210. Springer, 2006. doi: 10.1007/0-387-30528-9_7.

[25] Francesco Ceccon, John D. Siirola, and Ruth Misener. SUSPECT: MINLP special structure detector for Pyomo. *TOP*, 28:579–610, 2020. doi: 10.1007/s11750-020-00546-1.

[26] Federico Ceccon, Jordan Jalving, Joshua Haddad, Alexander Thebelt, Calvin Tsay, Carl D. Laird, and Ruth Misener. OMLT: Optimization & machine learning toolkit. *Journal of Machine Learning Research*, 23(349):1–8, 2022.

[27] Ross Anderson, Joey Huchette, Will Ma, Christian Tjandraatmadja, and Juan Pablo Vielma. Strong mixed-integer programming formulations for trained neural networks. In *International Conference on Integer Programming and Combinatorial Optimization*, pages 27–42. Springer, 2020. doi: 10.1007/978-3-030-45771-6_3.

[28] Harsha Nagarajan, Mowen Lu, Emre Yamangil, and Russell Bent. Tightening McCormick relaxations for nonlinear programs via dynamic multivariate partitioning. In *Principles and Practice of Constraint Programming (CP 2016)*, volume 9892 of *Lecture Notes in Computer Science*, pages 369–387. Springer, 2016. doi: 10.1007/978-3-319-44953-1_24.

[29] Harsha Nagarajan, Mowen Lu, Site Wang, Russell Bent, and Kaarthik Sundar. An adaptive, multivariate partitioning algorithm for global optimization of nonconvex programs. *Journal of Global Optimization*, 74(4):639–675, 2019. doi: 10.1007/s10898-018-00734-1.

[30] Federico Galvanin, Sandro Macchietto, and Fabrizio Bezzo. Model-based design of parallel experiments. *Industrial & Engineering Chemistry Research*, 46(3):871–882, 2007. doi: 10.1021/ie0611406.

[31] Marco Sandrin, Constantinos C. Pantelides, and Benoît Chachuat. Methodological and computational framework for model-based design of parallel experiment campaigns under uncertainty. *Journal of Process Control*, 152:103465, 2025. doi: 10.1016/j.jprocont.2025.103465.

[32] A. Raue, C. Kreutz, T. Maiwald, J. Bachmann, M. Schilling, U. Klingmüller, and J. Timmer. Structural and practical identifiability analysis of partially observed dynamical models by exploiting the profile likelihood. *Bioinformatics*, 25(15):1923–1929, 2009. doi: 10.1093/bioinformatics/btp358.

[33] K. Z. Yao, B. M. Shaw, B. Kou, K. B. McAuley, and D. W. Bacon. Modeling ethylene/butene copolymerization with multi-site catalysts: parameter estimability and experimental design. *Polymer Reaction Engineering*, 11(3):563–588, 2003. doi: 10.1081/PRE-120024426.

[34] R. Brun, P. Reichert, and H. R. Künsch. Practical identifiability analysis of large environmental simulation models. *Water Resources Research*, 37(4):1015–1030, 2001. doi: 10.1029/2000WR900350.

[35] William G. Hunter and Albey M. Reiner. Designs for discriminating between two rival models. *Technometrics*, 7(3):307–323, 1965. doi: 10.1080/00401706.1965.10490265.

[36] G. Buzzi-Ferraris, P. Forzatti, G. Emig, and H. Hofmann. Sequential experimental design for model discrimination in the case of multiple responses. *Chemical Engineering Science*, 39(1):81–85, 1984. doi: 10.1016/0009-2509(84)80132-8.

[37] Simon Olofsson, Marc Peter Deisenroth, and Ruth Misener. GPdoemd: A Python package for design of experiments for model discrimination. In *Computer Aided Chemical Engineering*, volume 46, pages 949–954. Elsevier, 2019. doi: 10.1016/B978-0-12-818634-3.50159-7.

[38] D. V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956. doi: 10.1214/aoms/1177728069.

[39] Adam Foster, Martin Jankowiak, Eli Bingham, Paul Horsfall, Yee Whye Teh, Tom Rainforth, and Noah Goodman. Variational Bayesian optimal experimental design. *Advances in Neural Information Processing Systems*, 32, 2019.

[40] A. C. Atkinson, B. Bogacka, and M. B. Bogacki. D- and T-optimum designs for the kinetics of a reversible chemical reaction. *Chemometrics and Intelligent Laboratory Systems*, 43(1–2):185–198, 1998. doi: 10.1016/S0169-7439(98)00046-X.

[41] George Mavrotas. Effective implementation of the ε-constraint method in multi-objective mathematical programming problems. *Applied Mathematics and Computation*, 213(2):455–465, 2009. doi: 10.1016/j.amc.2009.03.037.

[42] Indraneel Das and J. E. Dennis. Normal-boundary intersection: A new method for generating the Pareto surface in nonlinear multicriteria optimization problems. *SIAM Journal on Optimization*, 8(3):631–657, 1998. doi: 10.1137/S1052623496307510.

[43] A. Messac, A. Ismail-Yahaya, and C. A. Mattson. The normalized normal constraint method for generating the Pareto frontier. *Structural and Multidisciplinary Optimization*, 25(2):86–98, 2003. doi: 10.1007/s00158-002-0276-1.

[44] Eckart Zitzler and Lothar Thiele. Multiobjective optimization using evolutionary algorithms — a comparative case study. In *Parallel Problem Solving from Nature (PPSN V)*, volume 1498 of *Lecture Notes in Computer Science*, pages 292–301. Springer, 1998. doi: 10.1007/BFb0056872.

[45] Peter A. N. Bosman and Dirk Thierens. The balance between proximity and diversity in multiobjective evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 7(2):174–188, 2003. doi: 10.1109/TEVC.2003.810761.

[46] Hanif D. Sherali and Warren P. Adams. A hierarchy of relaxations between the continuous and convex hull representations for zero-one programming problems. *SIAM Journal on Discrete Mathematics*, 3(3):411–430, 1990. doi: 10.1137/0403036.

[47] Jean B. Lasserre. Global optimization with polynomials and the problem of moments. *SIAM Journal on Optimization*, 11(3):796–817, 2001. doi: 10.1137/S1052623400366802.

[48] Stefan Scholtes. Convergence properties of a regularization scheme for mathematical programs with complementarity constraints. *SIAM Journal on Optimization*, 11(4): 918–936, 2001. doi: 10.1137/S1052623499361233.

[49] E. M. L. Beale and J. A. Tomlin. Special facilities in a general mathematical programming system for non-convex problems using ordered sets of variables. *OR*, 69:447–454, 1970.

[50] Zhi-Quan Luo, Jong-Shi Pang, and Daniel Ralph. *Mathematical Programs with Equilibrium Constraints*. Cambridge University Press, 1996. doi: 10.1017/CBO9780511983658.

[51] Stephen Boyd, Seung-Jean Kim, Lieven Vandenberghe, and Arash Hassibi. A tutorial on geometric programming. *Optimization and Engineering*, 8(1):67–127, 2007. doi: 10.1007/s11081-007-9001-7.

[52] Costas D. Maranas and Christodoulos A. Floudas. Global optimization in generalized geometric programming. *Computers & Chemical Engineering*, 21(4):351–369, 1997. doi: 10.1016/S0098-1354(96)00282-7.

[53] John W. Chinneck and Erik W. Dravnieks. Locating minimal infeasible constraint sets in linear programs. *ORSA Journal on Computing*, 3(2):157–168, 1991. doi: 10.1287/ijoc.3.2.157.

[54] Tobias Achterberg. Conflict analysis in mixed integer programming. *Discrete Optimization*, 4(1):4–20, 2007. doi: 10.1016/j.disopt.2006.10.006.

[55] R. M. Van Slyke and Roger Wets. L-shaped linear programs with applications to optimal control and stochastic programming. *SIAM Journal on Applied Mathematics*, 17(4): 638–663, 1969. doi: 10.1137/0117061.

[56] R. T. Rockafellar and Roger J.-B. Wets. Scenarios and policy aggregation in optimization under uncertainty. *Mathematics of Operations Research*, 16(1):119–147, 1991. doi: 10.1287/moor.16.1.119.

[57] Wai-Kei Mak, David P. Morton, and R. Kevin Wood. Monte Carlo bounding techniques for determining solution quality in stochastic programs. *Operations Research Letters*, 24(1–2):47–56, 1999. doi: 10.1016/S0167-6377(98)00054-6.

[58] R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *The Journal of Risk*, 2(3):21–41, 2000. doi: 10.21314/JOR.2000.038.

[59] John R. Birge and François Louveaux. *Introduction to Stochastic Programming*. Springer Series in Operations Research and Financial Engineering. Springer New York, 2nd edition, 2011. doi: 10.1007/978-1-4614-0237-4.

[60] Benoît Colson, Patrice Marcotte, and Gilles Savard. An overview of bilevel optimization. *Annals of Operations Research*, 153(1):235–256, 2007. doi: 10.1007/s10479-007-0176-2.

[61] Pietro Belotti, Jon Lee, Leo Liberti, François Margot, and Andreas Wächter. Branching and bounds tightening techniques for non-convex MINLP. *Optimization Methods and Software*, 24(4–5):597–634, 2009. doi: 10.1080/10556780903087124.

[62] Ambros M. Gleixner, Timo Berthold, Benjamin Müller, and Stefan Weltge. Three enhancements for optimization-based bound tightening. *Journal of Global Optimization*, 67(4):731–757, 2017. doi: 10.1007/s10898-016-0450-4.

[63] Matteo Fischetti, Fred Glover, and Andrea Lodi. The feasibility pump. *Mathematical Programming*, 104(1):91–104, 2005. doi: 10.1007/s10107-004-0570-3.

[64] Marco A. Duran and Ignacio E. Grossmann. An outer-approximation algorithm for a class of mixed-integer nonlinear programs. *Mathematical Programming*, 36(3):307–339, 1986. doi: 10.1007/BF02592064.

[65] Roger Fletcher and Sven Leyffer. Solving mixed integer nonlinear programs by outer approximation. *Mathematical Programming*, 66(1–3):327–349, 1994. doi: 10.1007/BF01581153.

[66] Stephen J. Wright. *Primal-Dual Interior-Point Methods*. SIAM, 1997. ISBN 978-0-89871-382-4. doi: 10.1137/1.9781611971453.

[67] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.

[68] Andreas Wächter and Lorenz T. Biegler. Line search filter methods for nonlinear programming: Motivation and global convergence. *SIAM Journal on Optimization*, 16(1): 1–31, 2005. doi: 10.1137/S1052623403426556.

[69] Anthony V. Fiacco. *Introduction to Sensitivity and Stability Analysis in Nonlinear Programming*. Academic Press, 1983. ISBN 978-0-12-254450-7.

[70] Hans Pirnay, Rodrigo López-Negrete, and Lorenz T. Biegler. Optimal sensitivity based on IPOPT. *Mathematical Programming Computation*, 4(4):307–331, 2012. doi: 10.1007/s12532-012-0043-2.

[71] Metin Turkay and Ignacio E. Grossmann. Logic-based MINLP algorithms for the optimal synthesis of process networks. *Computers & Chemical Engineering*, 20(8):959–978, 1996. doi: 10.1016/0098-1354(95)00219-7.

[72] Lorenz T. Biegler. *Nonlinear Programming: Concepts, Algorithms, and Applications to Chemical Processes*. SIAM, 2010. ISBN 978-0-89871-702-0. doi: 10.1137/1.9780898719383.

[73] Janet E. Cuthrell and Lorenz T. Biegler. On the optimization of differential-algebraic process systems. *AIChE Journal*, 33(8):1257–1270, 1987. doi: 10.1002/aic.690330804.

[74] Aharon Ben-Tal and Arkadi Nemirovski. Robust solutions of uncertain linear programs. *Operations Research Letters*, 25(1):1–13, 1999. doi: 10.1016/S0167-6377(99)00016-4.

[75] Aharon Ben-Tal, Laurent El Ghaoui, and Arkadi Nemirovski. *Robust Optimization*. Princeton Series in Applied Mathematics. Princeton University Press, 2009. ISBN 978-0-691-14368-2.

[76] Dimitris Bertsimas and Melvyn Sim. The price of robustness. *Operations Research*, 52 (1):35–53, 2004. doi: 10.1287/opre.1030.0065.

[77] Aharon Ben-Tal, Alexander Goryashko, Elana Guslitzer, and Arkadi Nemirovski. Adjustable robust solutions of uncertain linear programs. *Mathematical Programming*, 99 (2):351–376, 2004. doi: 10.1007/s10107-003-0454-y.

[78] Xin Chen, Melvyn Sim, Peng Sun, and Jiawei Zhang. A linear decision-based approximation approach to stochastic programming. *Operations Research*, 56(2):344–357, 2008. doi: 10.1287/opre.1070.0457.

[79] Dimitris Bertsimas and Vineet Goyal. On the power and limitations of affine policies in two-stage adaptive optimization. *Mathematical Programming*, 134(2):491–531, 2012. doi: 10.1007/s10107-011-0444-4.

[80] PyO3 Contributors. PyO3: Rust bindings for the Python interpreter, 2024. URL https://github.com/PyO3/pyo3.

[81] Michael R. Bussieck, Arne Stolbjerg Drud, and Alexander Meeraus. MINLPLib — A collection of test models for mixed-integer nonlinear programming. *INFORMS Journal on Computing*, 15(1):114–119, 2003. doi: 10.1287/ijoc.15.1.114.15159.

[82] Philip E. Gill, Walter Murray, Michael A. Saunders, and Margaret H. Wright. A practical anti-cycling procedure for linearly constrained optimization. *Mathematical Programming*, 45(1–3):437–474, 1989. doi: 10.1007/BF01589114.

[83] Alexander D. Rikun. A convex envelope formula for multilinear functions. *Journal of Global Optimization*, 10(4):425–437, 1997. doi: 10.1023/A:1008217604285.

[84] Daniel K. Molzahn and Ian A. Hiskens. A survey of relaxations and approximations of the power flow equations. *Foundations and Trends in Electric Energy Systems*, 4(1–2): 1–221, 2019. doi: 10.1561/3100000012.

[85] C. A. Haverly. Studies of the behavior of recursion for the pooling problem. *ACM SIGMAP Bulletin*, (25):19–28, 1978. doi: 10.1145/1111237.1111238.

[86] Ruth Misener and Christodoulos A. Floudas. Global optimization of large-scale generalized pooling problems: Quadratically constrained MINLP models. *Industrial & Engineering Chemistry Research*, 49(11):5424–5438, 2010. doi: 10.1021/ie100025e.

[87] Emilie Danna, Edward Rothberg, and Claude Le Pape. Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming*, 102(1):71–90, 2005. doi: 10.1007/s10107-004-0518-7.

[88] Matteo Fischetti and Andrea Lodi. Local branching. *Mathematical Programming*, 98 (1–3):23–47, 2003. doi: 10.1007/s10107-003-0395-5.

[89] Yonathan Bard. *Nonlinear Parameter Estimation*. Academic Press, 1974.

[90] Gaia Franceschini and Sandro Macchietto. Model-based design of experiments for parameter precision: State of the art. *Chemical Engineering Science*, 63(19):4846–4872, 2008. doi: 10.1016/j.ces.2007.11.034.

[91] Jialu Wang and Alexander W. Dowling. Pyomo.DoE: An open-source package for model-based design of experiments in Python. *AIChE Journal*, 68(12):e17813, 2022. doi: 10.1002/aic.17813.

[92] Leonardo D. González and Victor M. Zavala. New paradigms for exploiting parallel experiments in Bayesian optimization. *Computers & Chemical Engineering*, 170:108110, 2023. doi: 10.1016/j.compchemeng.2022.108110.

[93] Yunfei Chu and Juergen Hahn. Parameter set selection for estimation of nonlinear dynamic systems. *AIChE Journal*, 53(11):2858–2870, 2007. doi: 10.1002/aic.11295.

[94] H. Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974. doi: 10.1109/TAC.1974.1100705.

[95] Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, Boston, 1999. doi: 10.1007/978-1-4615-5563-6.

[96] Matthias Ehrgott. *Multicriteria Optimization*. Springer, 2nd edition, 2005. doi: 10.1007/3-540-27659-9.

[97] Paula M. J. Harris. Pivot selection methods of the devex LP code. *Mathematical Programming*, 5(1):1–28, 1973. doi: 10.1007/BF01580108.